

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

[Why
HTML?](#)

[Report
Issue](#)

[Back to
Abstract](#)

[Download
PDF](#)

arXiv:2506.16121v1 [cs.DS] 19 Jun 2025

On the Efficient Discovery of Maximum k -Defective Biclique

Donghang Cui

Beijing Institute of Technology
cuidonghang@bit.edu.cn

Ronghua Li

Beijing Institute of Technology
lironghuabit@126.com

Qiangqiang Dai

Beijing Institute of Technology
qiangd66@gmail.com

Hongchao Qin

Beijing Institute of Technology
qhc.neu@gmail.com

Guoren Wang

Beijing Institute of Technology
wanggrbit@gmail.com

(2026)

Abstract.

The problem of identifying the maximum edge biclique in bipartite graphs has attracted considerable attention in bipartite graph analysis, with numerous real-world applications such as fraud detection, community detection, and online recommendation systems. However, real-world graphs may contain noise or incomplete information, leading to overly restrictive conditions when employing the biclique model. To mitigate this, we focus on a new relaxed subgraph model, called the k -defective biclique, which allows for up to k missing edges compared to the biclique model. We investigate the problem of finding the maximum edge k -defective biclique in a bipartite graph, and prove that the problem is NP-

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why
HTML?

Report
Issue

Back to
Abstract

Download
PDF

Extensive experiments on 10 large real-world datasets validate the efficiency and effectiveness of the proposed approaches. The results indicate that our algorithms consistently outperform state-of-the-art algorithms, offering up to $1000\times$ speedups across various parameter settings.

±

†copyright: none

±

†copyright: acmlicensed

±

†journalyear: 2026

±

†doi: XXXXXXXX.XXXXXXX

±

†conference: -, May 31–June 05, 2026; Bengaluru, India

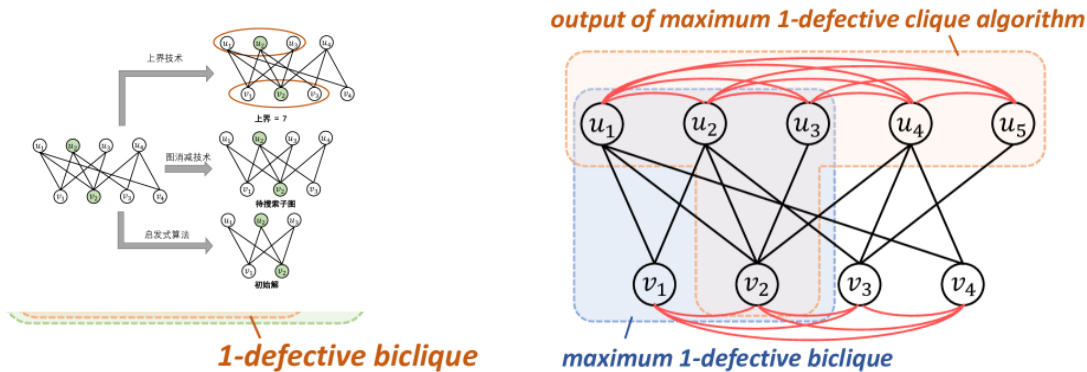
1. Introduction

Bipartite graph is a commonly encountered graph structure consisting of two disjoint vertex sets and an edge set, where each edge connects vertices from the two different sets. In recent decades, bipartite graphs have been utilized as a fundamental tool for modeling various binary relationships, such as user-product interactions in e-commerce (Li et al., 2023), authorship connections between authors and publications (Ley, 2002), and associations between genes and phenotypes in bioinformatics (Prelić et al., 2006; Eren et al., 2013). Mining cohesive subgraphs from bipartite graphs has proven to be critically important across diverse domains (Dai et al., 2023b; Wang et al., 2021). For instance, identifying cohesive subgraphs in reviewer-product graphs facilitates the identification of fraudulent reviews or camouflage attacks (Hooi et al., 2017).

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

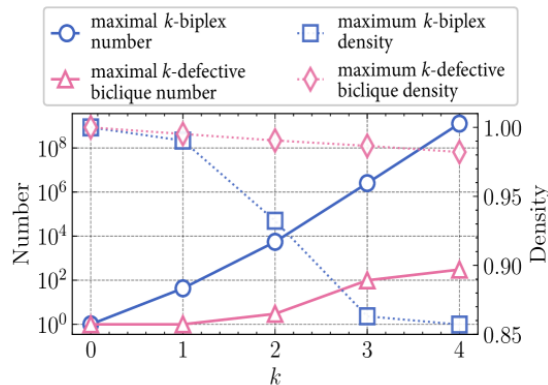
Why Report Back to Download
HTML? Issue Abstract PDF

Lyu et al., 2020). However, real-world graphs often contain noisy or incomplete information (Li et al., 2008), making the biclique model overly restrictive for capturing cohesive subgraphs on such a kind of bipartite graphs. More relaxed biclique models offer a promising alternative to address this challenge. The k -biplex model, which allows each vertex to have at most k non-neighbors, has recently attracted significant attention (Dai et al., 2023b; Yu and Long, 2023; Yu et al., 2022). However, due to its excessive structural relaxation, a slightly increase in k can lead to: (1) a sharp rise in quantity, potentially reducing the mining efficiency; and (2) a significant decrease in density, undermining the capability to represent cohesive communities.



(a) Example graph showcasing a biclique and two k -defective cliques

(b) Example of incorrect output from k -defective clique algorithms



(c) Comparison of quantity and density between k -biplex and k -defective

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

Figure 1. Illustrations of k -defective biclique, examples and comparisons

Inspired by the widely studied k -defective clique in traditional graphs (Yu et al., 2006; Chen et al., 2021b; Dai et al., 2024a), we introduce a novel cohesive subgraph model for bipartite graphs, named k -defective biclique. Specifically, a k -defective biclique is a subgraph of a bipartite graph containing at most k missing edges. Fig. 1a illustrates an example graph showcasing two k -defective bicliques, where the 1-defective biclique has one missing edge (u_3, v_1) , and the 3-defective biclique has three missing edges (u_1, v_3) , (u_3, v_1) and (u_3, v_3) .

The k -defective biclique offers greater flexibility over biclique, as its density can be controlled by adjusting k to suit specific requirements. When $k = 0$, the k -defective biclique degenerates to the biclique, thereby illustrating its generalization capability. In comparison to the k -biplex, k -defective biclique offers a more fine-grained relaxation of the biclique due to its stricter retention of edges. As illustrated in Fig. 1c, the k -defective biclique exhibits much closer quantity and density to the biclique compared to k -biplex, especially for larger k . Here, we refer to a k -defective biclique as "maximal" if it cannot be contained within any other k -defective biclique (or k -biplex), and as "maximum" when it is the maximal k -defective biclique (or k -biplex) with the largest number of edges. In this paper, we focus mainly on the problem of identifying a maximum k -defective biclique. We prove that the maximum k -defective biclique search problem is NP-hard, indicating that no polynomial-time algorithms exist for solving this problem unless $NP = P$.

Notably, adapting existing maximum/maximal k -defective clique algorithms (Chang, 2024; Dai et al., 2024a) and maximum biclique algorithms (Lyu et al., 2020; Gillis and Glineur, 2014) to our problem poses significant challenges for the following reasons: (1) Adapting the maximum k -defective clique algorithm involves fully connecting vertices on each side of the bipartite graph (as shown in Fig. 1b). However, applying such algorithms to these graphs may yield incorrect results. For instance, in Fig. 1b, the maximum 1-defective biclique has 5 edges whereas the output of maximum 1-defective clique algorithms yields only 4 edges; (2) Applying maximal k -defective clique algorithms on such each-side-fully-connected graphs will generate a large number of redundant branches and is computationally expensive (as evidenced in Table 2); and (3) Adapting maximum biclique algo-

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why
HTML?

Report
Issue

Back to
Abstract

Download
PDF

Contributions. To efficiently address the maximum k -defective biclique search problem, we conduct an in-depth study of its properties and develop two novel algorithms with the worst-case time complexity less than $O^*(2^n)$. We also present a series of optimization techniques to further improve the efficiency of our algorithms. Finally, we evaluate the effectiveness and efficiency of our methods through comprehensive experiments. The main contributions of this paper are summarized as follows:

A new cohesive subgraph model. We propose a new cohesive subgraph model for bipartite graphs named the k -defective biclique, which is defined as a subgraph with at most k missing edges. We then demonstrate that the problem of finding the maximum k -defective biclique in bipartite graphs is NP-hard.

Two Novel Search Algorithms. We propose two novel maximum k -defective biclique search algorithms with nontrivial worst-case time complexity guarantees. The first algorithm utilizes a branch-and-bound technique combined with a newly-developed binary branching strategy, which has a time complexity of $O(m\alpha_k^n)$, where n and m represent the number of vertices and edges of the bipartite graph respectively, and α_k is a positive real number strictly less than 2 (e.g., for $k = 1, 2, 3$, $\alpha_k = 1.911, 1.979$, and 1.995 , respectively). The second algorithm employs an innovative pivoting-based branching strategy that effectively integrates with our binary branching strategy, which achieves a lower time complexity of $O(m\beta_k^n)$, where β_k is a positive real number strictly less than α_k . (e.g., when $k = 1, 2, 3$, $\beta_k = 1.717, 1.856$, and 1.931 , respectively).

Nontrivial optimization techniques. We also develop a series of nontrivial optimization techniques to improve the efficiency of our proposed algorithms, including graph reduction techniques, novel upper-bound techniques, and a heuristic approach. Specifically, the graph reduction techniques focus on reducing the size of the input bipartite graph based on constraints, such as the core and the number of common neighbors. We devise tight vertex and edge upper bounds to minimize unnecessary branching, and then design an efficient algorithm to compute the proposed upper bounds with linear time complexity. Additionally, we propose a heuristic approach by incrementally ex-

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

effectiveness, and scalability of our algorithms on 11 large real-world graphs. The experimental results show that our algorithms consistently outperform the state-of-the-art baseline method (Dai et al., 2023a) across all parameter settings. In particular, our algorithm achieves improvements of up to three orders of magnitude on most datasets. For instance, on the *Twitter* (9M vertices, 10M edges) dataset with $k = 2$, our algorithm takes 10 seconds while the baseline method cannot finish within 3 hours. Additionally, we perform a case study on a fraud detection task to show the high effectiveness of our solutions in real-world applications.

2. Preliminaries

Let $G = (U, V, E)$ be an undirected bipartite graph, where U and V are two disjoint sets of vertices, and $E \subseteq U \times V$ is the set of edges. Denote by $n = |U| + |V|$ and $m = |E|$ the total number of vertices and edges in G , respectively. Denote by $S = (U_S, V_S)$ a vertex subset of G , where $U_S \subseteq U$ and $V_S \subseteq V$. Let $v \in S$ indicate that $v \in U_S \cup V_S$. Let $G(S) = (U_S, V_S, E_S)$ be the subgraph of G induced by S , where $E_S = \{(u, v) | u \in U_S \wedge v \in V_S \wedge (u, v) \in E\}$. For $u \in U$, denote by $N(u) = \{v \in V | (u, v) \in E\}$ and $\bar{N}(u) = \{v \in V | (u, v) \notin E\}$ the set of neighbors and non-neighbors of u in G , respectively. Similar definitions apply for the vertices in U . Let $d(u) = |N(u)|$ and $\bar{d}(u) = |\bar{N}(u)|$ be the degree and non-degree of u in G , respectively. For a vertex subset (or a subgraph) S of G , let $N_S(u) = N(u) \cap (U_S \cup V_S)$ and $\bar{N}_S(u) = \{v \in S \setminus N(u) | u \text{ and } v \text{ are on opposite sides}\}$ be the set of u 's neighbors and non-neighbors in S , respectively. Let $d_S(u) = |N_S(u)|$ and $\bar{d}_S(u) = |\bar{N}_S(u)|$ be the degree and non-degree of u in S , respectively. For a bipartite graph $G = (U, V, E)$, we refer to $(u, v) \in U \times V$ as a non-edge of G if $(u, v) \notin E$. We denote $\bar{E}_S$ the set of non-edges in $G(S)$, i.e., $\bar{E}_S = U_S \times V_S \setminus E_S$. The formal definition of k -defective biclique is given below.

Definition 1 (k -defective biclique).

Given a bipartite graph $G = (U, V, E)$, a k -defective biclique $D = (U_D, V_D, E_D)$ of G is a subgraph of G that has at most k non-edges, i.e., $|U_D \times V_D \setminus E_D| \leq k$.

The k -defective biclique corresponds to the biclique if $k = 0$, which indicates that k -defective biclique represents a more general structure. If a k -defective biclique of G contains the most number

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

observe that a k -MDBs in many real-world graphs may exhibit a skewed structure, similar to that of maximum bicliques, where one side has a large number of vertices while the other side may contain only a few (or even one) vertices. Such k -MDBs often hold limited practical values in network analysis. To address this issue, we define a size threshold θ , representing the minimum number of vertices required on both sides of the k -MDB. Furthermore, we impose the condition $\theta > k$ to ensure that the k -MDB is connected, as demonstrated in the following lemma.

LEMMA 0.

Given a bipartite graph $G = (U, V, E)$ and a k -defective biclique $D = (U_D, V_D, E_D)$ of G , if $|U_D| > k$ and $|V_D| > k$, we establish that D is a connected subgraph in G .

Proof.

Assume that a k -defective biclique D is not connected when $|U_D| > k$ and $|V_D| > k$. In this case, D can be decomposed into two nonempty subgraphs $D_1 = (U_{D_1}, V_{D_1}, E_{D_1})$ and $D_2 = (U_{D_2}, V_{D_2}, E_{D_2})$, with the results of $E_{D_1} \cup E_{D_2} = E_D$. The number of non-edges between D_1 and D_2 is given by $|U_{D_1}| \cdot |V_{D_2}| + |U_{D_2}| \cdot |V_{D_1}|$, which exceeds $|U_D|$ (or $|V_D|$) and is thus larger than k . This contradicts the assumption that D is a k -defective biclique. Therefore, the proof is complete. ■

Based on Lemma 2, we formally define the problem addressed in this paper as follows.

Problem statement 0 (MDB search problem).

Given a bipartite graph $G = (U, V, E)$ and two integers k and θ with $\theta > k$, the MDB search problem aims to find a connected k -MDB $D = (U_D, V_D, E_D)$ of G satisfying $|U_D| \geq \theta$ and $|V_D| \geq \theta$.

THEOREM 3.

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

we establish the NP-hardness of the MDB search problem through a polynomial-time reduction from the classical NP-complete maximum clique search problem. Due to space limitation, we present the complete version of this proof and all the omitted proofs of this paper in (RNO, [line](#)). ■

3. Our Proposed Algorithms

In this section, we propose two novel algorithms to solve the k -MDB search problem. Specifically, we first propose a new branch-and-bound algorithm that utilizes a newly-designed binary branching strategy, achieving the worst-case time complexity of $O(m\alpha_k^n)$, where $\alpha_k < 2$ (e.g. $k = 1, 2$, and 3 , we have $\alpha_k = 1.911, 1.979$, and 1.995 , respectively). To further improve the efficiency, we propose a novel pivoting-based algorithm that incorporates a well-designed pivoting-based branching strategy with our proposed binary branching strategy. We prove that the time complexity of the improved algorithm is bounded by $O(m\beta_k^n)$, where $\beta_k < \alpha_k$ (e.g., when $k = 1, 2, 3$, $\beta_k = 1.717, 1.856$, and 1.931 , respectively). To the best of our knowledge, the algorithms we proposed in this section are the first two k -MDB search algorithms that surpass the trivial time complexity of $O^*(2^n)$.

3.1. A New Branch-and-bound Algorithm

We firstly introduce a branch-and-bound algorithm for the k -MDB problem. It employs a straightforward branch-and-bound approach by selecting a vertex v from the graph and dividing the k -MDB search problem into two subproblems: one seeks a k -MDB that includes v , while the other seeks a k -MDB that excludes v . Specifically, given a bipartite graph G , denote by $S = (U_S, V_S)$ a partial set and $C = (U_C, V_C)$ a candidate set, where $G(S)$ is a k -defective biclique of G , and for any vertex $u \in C$, $G(S \cup \{u\})$ forms a larger k -defective biclique of G . (S, C) is referred to as an instance, which aims to identify a k -MDB consisting of S and a subset of C . In particular, the instance $((\emptyset, \emptyset), (U, V))$ aims to find a k -MDB of G , while the instance $(S^*, (\emptyset, \emptyset))$ corresponds to a possible solution $G(S^*)$. To clarify, we refer to a vertex $u \in C$ as a *branching vertex* when a new sub-instance is generated by expanding S with u .

Given an instance (S, C) , our branch-and-bound algorithm selects a branching vertex $u \in C$ and generates two new sub-instances: $(S \cup \{u\}, C \setminus \{u\})$ and $(S, C \setminus \{u\})$. The algorithm then recursively

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

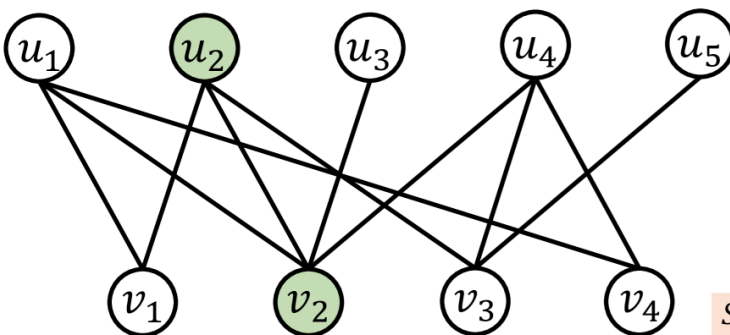
reduce the time complexity, we propose a new binary branching strategy.

Intuitively, if k vertices from C with non-neighbors in S are successively added to S , then S will contain at least k non-edges. Consequently, this implies that all vertices in C must be the neighbors of each vertex in S . Additionally, if no vertex in C that has non-neighbors in S , adding a vertex v from C to S will result in exactly $\bar{d}_C(v)$ vertices in C with non-neighbors in S . Based on these observations, we give our new binary branching strategy as follows.

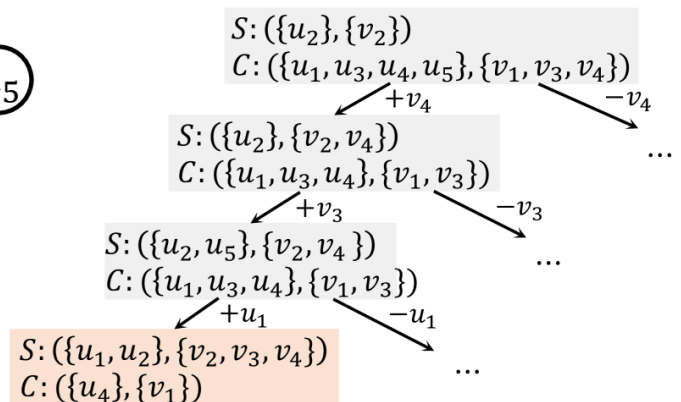
New binary branching strategy. Given an instance (S, C) , let $u = \arg \max_{u \in C} \bar{d}_S(u)$. The sub-instances are generated in the following way:

If $\bar{d}_S(u) > 0$, select u as the branching vertex and generate two sub-instances: $(S \cup \{u\}, C \setminus \{u\})$, $(S, C \setminus \{u\})$.

Otherwise, select $v \in C$ with the maximum $\bar{d}_C(v)$ as the branching vertex, and generate two sub-instances: $(S \cup \{v\}, C \setminus \{v\})$, $(S, C \setminus \{v\})$.



(a) the input graph



(b) the branching tree

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

Example 1.

Fig. 2 shows an example of our new binary branching strategy for finding a 2-MDB. The input graph is depicted in Fig. 2a, where the current instance is $S = (\{u_2\}, \{v_2\})$ (green vertices) and $C = (\{u_1, u_3, u_4, u_5\}, \{v_1, v_3, v_4\})$ (uncolored vertices). Fig. 2(b) illustrates the branching tree obtained by applying our binary branching strategy. Specifically, in the top-level branch, v_4 is selected as the branching vertex due to its maximum number of non-neighbors in S . When v_4 is added to S , it brings a non-edge in $S \cup \{v_4\}$. Then, we observe that there is no vertex in the second-level branch with non-neighbors in $S \cup \{v_4\}$. Therefore, v_3 is selected as the second-level branching vertex as it has the most non-neighbors in $C \setminus \{v_4\}$. In the third-level branch, u_1 is selected as the branching vertex as it has the most non-neighbors in $S \cup \{v_4, v_3\}$. In the forth-level, $S \cup \{u_1, v_3, v_4\}$ contains 2 non-edges, leading to the deletion of u_3 from C since it still has non-neighbors in $S \cup \{u_1, v_3, v_4\}$, which is the key to reducing the unnecessary computations.

Implementation details 0.

Algorithm 1 outlines the pseudocode of our proposed branch-and-bound algorithm. It takes a bipartite graph G and two integers k and θ as input parameters, and returns a k -MDB D^* of G . It first initializes D^* as an empty graph, and then starts to search k -MDB by invoking the procedure `BranchB` where all vertices in G serve as the candidate set (line 2). `BranchB` holds an instance (S, C) as its parameter, where S denotes the partial set and C denotes the candidate set. Within `BranchB`, the algorithm first determines whether a larger k -defective biclique has been found (lines 5-8). After that, it selects a branching vertex u from C using the new binary branching strategy (lines 9-10) and generates two sub-instance $(S', C' \cup \{u\})$ and $(S, C \setminus \{u\})$ (lines 11-12). Here, S' and C' are the new partial set and new candidate set derived by invoking `Update` (line 11), ensuring the candidate set keeps all valid branching vertex (lines 14-15). We next analyze the time complexity of Algorithm 1.

Input: Bipartite graph $G = (U, V, E)$, integers k and θ

Output: A k -MDB of G

```

1 |
2 |  $D^* \leftarrow$  an empty graph;
3 | BranchB $((\emptyset, \emptyset), (U, V))$ ;
4 | return  $D^*$ ;
```

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why
HTML?

Report
Issue

Back to
Abstract

Download
PDF

```

1
1
1
2
1
3   $u \leftarrow \arg \max_{v \in C} \bar{d}_S(v);$ 
1
4
1
5  if  $\bar{d}_S(u) = 0$  then  $u \leftarrow \arg \max_{v \in C} \bar{d}_C(v);$ 
1
6
1
7   $(s', c') \leftarrow \text{Update}(S, C, u);$ 
1
8   $\text{BranchB}(S' \cup \{u\}, C'); \text{BranchB}(S, C \setminus \{u\});$ 
1
9
2
0
2
1 Function  $\text{Update}(S, C, u):$ 
2
2   $c' \leftarrow \{v \in C \setminus \{u\} \mid \bar{d}_{S \cup \{u\}}(v) \leq k - \bar{d}(S)\};$ 
2
3   $c'_0 \leftarrow \{v \in C' \mid \bar{d}_{S \cup \{u\} \cup C'}(v) = 0\};$ 
2
4  return  $(S \cup c'_0, C' \setminus c'_0);$ 
2
5
2
6

```

Algorithm 1 The branch-and-bound algorithm

THEOREM 2.

Given a bipartite graph $G = (U, V, E)$ and an integer k , the time complexity Algorithm 1 is given by $O(m\alpha_k^n)$, where α_k is the largest real root of equation $x^{2k+5} - 2x^{2k+4} + x^3 - x^2 + 1 = 0$. Specifically, when $k = 1, 2$ and 3 , we have $\alpha_k = 1.911, 1.979$ and 1.995 , respectively.

Proof.

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

Then, we analyze the size of $T(n)$ based on the binary branching strategy. In each branch of **BranchB**, either a vertex with at least one non-neighbor in S or a vertex with the most non-neighbors in C is selected as the branching vertex. If the vertex u with at least one non-neighbor in S is selected, it results in at least one additional non-edge in $S \cup \{u\}$. Alternatively, if a vertex v with the most non-neighbors in C is selected as the branching vertex, the next-level branch will contain at least $\bar{d}_C(v)$ vertices in C that have non-neighbors in $S \cup \{v\}$, where $\bar{d}_C(v) > 0$. Consequently, we can conclude that in the worst case, adding two vertices from C to S will result in at least one additional non-edge in S . After adding at most $2k + 1$ vertices from C to S , we have $|\bar{E}_S| = k$ and there exists a vertex $v \in C$ such that $\bar{d}_S(v) > 0$. Based on this, the recurrence is given by $T(n) \leq \sum_{i=1}^{2k+1} T(n-i) + T(n-2k-2)$.

Note that the $(2k + 2)$ -th level of branching (the term $T(n - 2k - 2)$) corresponds to a branch that searches for the maximum biclique. In this case, it is evident that any vertex in C selected as the branching vertex will result in at least one additional vertex being removed from C . Thus, we yield a recurrence of $T(n - 2k - 2) = T(n - 2k - 3) + T(n - 2k - 4)$ for such a branch. Based on above discussion, we can present the following tighter recurrence.

$$(1) \quad T(n) \leq \sum_{i=1}^{2k+1} T(n-i) + T(n-2k-3) + T(n-2k-4).$$

According to the theory proposed in (Fomin and Kaski, 2013), $T(n)$ can be expressed in the form of α_k^n , where the value of α_k derives from the largest real root of equation $x^n = x^{n-1} + x^{n-2} + \dots + x^{n-2k-1} + x^{n-2k-3} + x^{n-2k-4}$, which can be simplified to $x^{2k+5} - 2x^{2k+4} + x^3 - x^2 + 1 = 0$. Therefore, the theorem is proven. ■

LEMMA 0.

Given a bipartite graph $G = (U, V, E)$, the time complexity of Algorithm 1 for finding a 0-MDB of G is $O(m \times 1.618^n)$.

Proof.

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

3.2. A Novel Pivoting-based Algorithm

We observe that Algorithm 1 may still generate numerous unnecessary sub-instances. Consider an instance (S, C) and a branching vertex $v \in C$ with $\bar{d}_S(v) = 0$. In Algorithm 1, if v and all non-neighbors of v have been excluded from the current instance, the remaining vertices cannot form any k -MDB. This is because any k -defective biclique derived from these remaining vertices can always be expanded by including v . Consecutively, Algorithm 1 may produce many k -defective bicliques that are not maximum. To address the problem and further improve the efficiency, we propose a novel pivoting-based algorithm in this section. We formalize above insights into the following lemma.

LEMMA 0.

Given an instance (S, C) and any vertex $v \in C$ with $\bar{d}_S(v) = 0$, the k -MDB of instance (S, C) must contain either v or a vertex in $\bar{N}_C(v)$.

Based on Lemma 4, we develop a pivoting strategy that provides a new way to generate sub-instances. The details are presented as follows.

Pivoting strategy. Consider an instance (S, C) and a vertex $u \in C$ with $\bar{d}_S(u) = 0$. Let u be the *pivot vertex*. Assume $\bar{N}_C(u) = \{v_1, v_2, \dots, v_r\}$, where $r = \bar{d}_C(u)$. The pivoting strategy treats each vertex in $\{u\} \cup \bar{N}_C(u)$ as a branching vertex and generates exactly $r + 1$ sub-instances. The first sub-instance is given by $(S \cup \{u\}, C \setminus \{u\})$, and for $i \geq 2$, the i -th sub-instance is given by $(S \cup \{v_{i-1}\}, C \setminus \{u, v_1, v_2, \dots, v_{i-1}\})$.

With this strategy, S is expanded only with the vertex u and vertices in $\bar{N}_C(u)$, avoiding redundant sub-instances that would lead to non-maximum k -defective biclique. Next, we consider the selection of the pivot vertex. Let $C_0 \subseteq C$ be the set of vertices fully connected to S , i.e., $C_0 = \{v \in C \mid \bar{d}_S(v) = 0\}$. According to the pivoting strategy, any vertex from C_0 can serve as a pivot

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

oting strategy inapplicable. Fortunately, as discussed in Sec. 3.1, continuously branching k times by adding vertices from C to S will result in S containing at least k non-edges. Moreover, C becomes an empty set after this process, which significantly reduces unnecessary computations. Motivated by this observation, we integrate the pivoting strategy with the binary branching strategy to develop a novel pivoting-based branching strategy as outlined below.

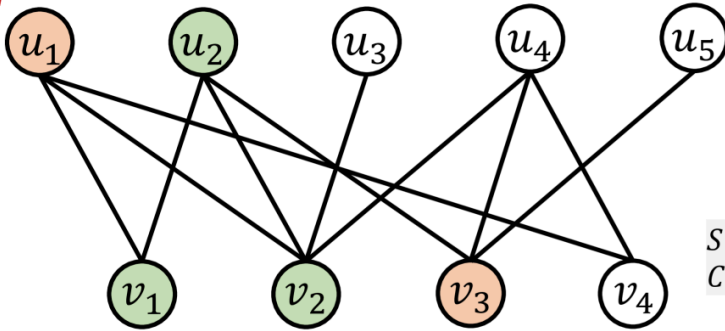
Novel pivoting-based branching strategy. Given an instance (S, C) , let $C_0 = \{v \in C \mid \bar{d}_S(v) = 0\}$. The new sub-instances of (S, C) are generated in the following way:

If C_0 is empty or $|C \setminus C_0| > k - |\bar{E}_S|$, let $u = \arg \max_{v \in C} \bar{d}_S(v)$ be the branching vertex and generate two sub-instances: $(S \cup \{u\}, C \setminus \{u\})$ and $(S, C \setminus \{u\})$.

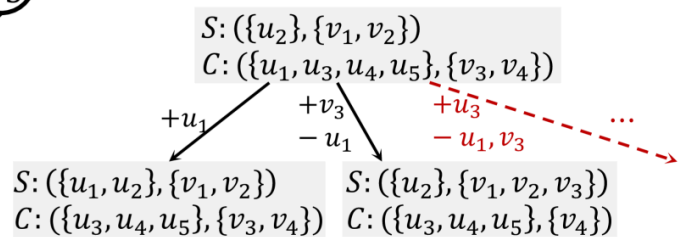
Otherwise, let $u = \arg \min_{v \in C_0} \bar{d}_C(v)$, and then:

- If $\bar{d}_C(u) > k - |\bar{E}_S| > 0$, generate two sub-instances with u as the branching vertex: $(S \cup \{u\}, C \setminus \{u\})$ and $(S, C \setminus \{u\})$.
- Otherwise, assume $\bar{N}_C(u) = \{v_1, v_2, \dots, v_r\}$, where $r = \bar{d}_C(u)$, and generate $\bar{d}_C(u) + 1$ sub-instances with u as the pivot vertex, where the first sub-instance is given by $(S \cup \{u\}, C \setminus \{u\})$, and for $i \geq 2$, the i -th sub-instance is given by $(S \cup \{v_{i-1}\}, C \setminus \{u, v_1, v_2, \dots, v_{i-1}\})$.

pivot



(a) the input graph



(b) the branching tree

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

Example 5.

Fig. 3 shows an example of the our pivoting-based branching strategy for the 2-MDB search. The example graph is given by Fig. 3a, with the current instance set as $S = (\{u_2\}, \{v_1, v_2\})$ (green vertices) and $C = (\{u_1, u_3, u_4, u_5\}, \{v_3, v_4\})$ (orange and uncolored vertices). According to the pivoting-based branching strategy, $C_0 = (\{u_1\}, \{v_3\})$, and u_1 is selected as the pivot vertex since it has the fewest non-neighbors in C among vertices in C_0 . Then, only u_1 and all non-neighbors of u_1 in C are used to generate new sub-instances (orange vertices), while other vertices of C are not considered (uncolored vertices). This results in only two sub-instances generated by our pivot-based branching strategy. The branching tree for this example is illustrated in Fig. 3b, where the black arrows represent the generated branches while the red arrows indicate the unnecessary branches.

Implementation details 0.

Algorithm 2 outlines the pseudocode of the proposed pivot-based algorithm. It begins by searching for k -MDB by invoking the procedure BranchP with an empty partial set S and the entire vertex set of G as the candidate set C (line 2). The algorithm returns the global variable D^* as the final k -MDB after BranchP is processed (line 3). Within BranchP, the algorithm first updates D^* if a larger k -defective biclique is found (lines 5-7). Then, it generates new sub-instances based on the pivoting-based branching strategy (lines 8-22). Specifically, if $C_0 = \emptyset$ or $|C \setminus C_0| > k - |\bar{E}_S|$, the vertex $u = \arg \max_{v \in C} \bar{d}_S(v)$ is selected as the branching vertex (line 11), and two new sub-instances are generated (line 13). Otherwise, the algorithm assigns $u = \arg \min_{v \in C_0} \bar{d}_C(v)$. If $\bar{d}_C(u) > k - |\bar{E}_S|$, two sub-instances are generated with u as the branching vertex (lines 16-18). Otherwise, $\bar{d}_C(u) + 1$ new sub-instances are generated with u and all u 's non-neighbors in C as branching vertex (lines 20-22). When a branching vertex is added to S , the algorithm adopts the Update procedure presented in Algorithm 1 to update the current instance (lines 12, 17 and 21). Below, we analyze the time complexity of Algorithm 2.

Input: Bipartite graph $G = (U, V, E)$, integers k and θ

Output: A k -MDB of G

```

1 |
2 |  $D^* \leftarrow$  an empty graph;
3 | BranchP( $(\emptyset, \emptyset), (U, V)$ );
```


This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

[Why
HTML?](#)

[Report
Issue](#)

[Back to
Abstract](#)

[Download
PDF](#)

```

0      return ;
1
1
1
2
1
3       $C_0 \leftarrow \{v \in C \mid \tilde{d}_S(v) = 0\};$ 
4
4
1
5      if  $C_0 = \emptyset$  or  $|C \setminus C_0| > k - |\tilde{E}_S|$  then
1
6           $u \leftarrow \arg \max_{v \in C} \tilde{d}_S(v);$ 
1
7           $(s', c') \leftarrow \text{Update}(S, C, u);$ 
1
8           $\text{BranchP}(S' \cup \{u\}, C'); \text{BranchP}(S, C \setminus \{u\});$ 
1
9
2
0      else
2
1           $u \leftarrow \arg \min_{v \in C_0} \tilde{d}_C(v);$ 
2
2          if  $\tilde{d}_C(u) > k - |\tilde{E}_S| > 0$  then
2
3               $(s', c') \leftarrow \text{Update}(S, C, u);$ 
2
4               $\text{BranchP}(S' \cup \{u\}, C'); \text{BranchP}(S, C \setminus \{u\});$ 
2
5
2
6      else
2
7          for  $v \in \{u\} \cup \tilde{N}_C(u)$  do
2
8               $(s', c') \leftarrow \text{Update}(S, C, v);$ 
2
9               $\text{BranchP}(S' \cup \{v\}, C'); c \leftarrow c \setminus \{v\};$ 
3
0
3
1
3
2
3
3
3
4

```

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

Given a bipartite graph $G = (U, V, E)$ and an integer k , the time complexity of Algorithm 2 is given by $O(m\beta_k^n)$, where β_k is the largest real root of equation $x^{2k+5} - 2x^{2k+4} + x^{k+3} - 2x + 2 = 0$. Specifically, when $k = 1, 2$ and 3 , we have $\beta_k = 1.717, 1.856$ and 1.931 , respectively.

Proof.

In Algorithm 2, Let $T(n)$ denote the maximum number of leave instances generated by BranchP, where $n = |C| = |U_C| + |V_C|$. It is easy to verify that finding the vertex with most (least) non-neighbors in S (C) takes at most $O(m)$ time. Then, each recursive call of BranchP takes $O(m)$ time, and thus BranchP takes $O(m \cdot T(n))$ time. We next focus on analyzing the size of $T(n)$. Based on the pivoting-based branching strategy, there are three cases that need to be discussed:

- 1) If $C_0 = \emptyset$ or $|C \setminus C_0| > k - |\bar{E}(S)|$, BranchP continuously applies the binary branching strategy. After moving at most k vertices from C to S , S contains k non-edges. At this point, at least one vertex in C will be removed as it has non-neighbors in S . Therefore, the recurrence for this case is given by $T(n) \leq \sum_{i=1}^{k+1} T(n-i)$.
- 2) If $k - |\bar{E}_S| = 0$, BranchP degenerates into maximum biclique search and continuously applies the pivoting strategy. Let $r = \bar{d}_C(u)$, the recurrence for this case is given by $T(n) \leq T(n-r-1) + \sum_{i=1}^r T(n-r-i)$. If $r \leq 1$, we have $T(n) \leq 2T(n-2)$. If $r \geq 2$, let $T(n) = \gamma_r^n$ according to the existing theory (Fomin and Kaski, 2013), where γ_r is the largest real root of equation $x^n = x^{n-r-1} + \sum_{i=1}^r x^{n-r-i}$. The value of γ_r is monotonically decreasing with respect to r when $r \geq 2$. By solving the equation we have $\gamma_2 = 1.395 < \sqrt{2}$. As a result, we can always say $T(n) \leq 2T(n-2)$ for this case.
- 3) Otherwise, let $u = \arg \min_{v \in C_0} \bar{d}_C(v)$. If $\bar{d}_C(u) \leq k - |\bar{E}(S)|$, BranchP performs the pivoting strategy with u as the pivot vertex. This strategy generates at most $\bar{d}_C(u) + 1 \leq k + 1$ sub-instances. Therefore, the recurrence for this case is given by $T(n) \leq \sum_{i=1}^{k+1} T(n-i)$. Otherwise, BranchP uses u to perform binary branching strategy. In the next-level branching, we have $|C \setminus C_0| > k - |\bar{E}(S \cup \{u\})|$, as $\bar{d}_C(u) > k - |\bar{E}(S)|$. In this case, the sub-branch (the term $T(n-1)$) will continuously perform the binary branching strategy as discussed in case (1). Consequently, the recurrence for this case is given by $T(n) \leq \sum_{i=1}^{k+1} T(n-i) + T(n-2k-2)$. Note that the term $T(n-2k-2)$ degenerates to maximum biclique search, and we can derive that $T(n-2k-2) = 2T(n-2k-4)$ as discussed in case (2). Therefore, the final recurrence for this case is given by $T(n) \leq \sum_{i=1}^{k+1} T(n-i) + 2T(n-2k-4)$.

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why
HTML?

Report
Issue

Back to
Abstract

Download
PDF

Based on the existing theorem proposed in (Fomin and Kaski, [2013](#)), we denote by $T(n) = \beta_k^n$, where β_k is the largest real root of equation $x^n = x^{n-1} + x^{n-2} + \dots + x^{n-k-1} + 2x^{n-2k-4}$. The equation is equivalent to $x^{2k+5} - 2x^{2k+4} + x^{k+3} - 2x + 2 = 0$. Consequently, the theorem is proven. ■

LEMMA 0.

Given a bipartite graph $G = (U, V, E)$, the time complexity of Algorithm [2](#) for finding a 0-MDB of G is $O(m \cdot 1.414^n)$.

Proof.

When setting $k = 0$, Algorithm [2](#) will always execute the pivoting-based branching strategy (lines 20-22), corresponding to a recurrence of $T(n) < 2T(n-2)$ according to the proof of Theorem [6](#). Therefore, we have $T(n) = 1.414^n$. ■

Discussion. It is worth noting that the branch-and-bound and pivoting techniques has been widely applied on many cohesive subgraph models such as the k -defective clique (Dai et al., [2024a](#); Chang, [2023](#), [2024](#)), the k -biplex (Yu and Long, [2023](#); Yu et al., [2022](#)) and the biclique (Abidi et al., [2020](#); Alexe et al., [2004](#); Chen et al., [2022](#)). However, our binary branching strategy and pivoting-based branching strategy are significantly different from all these existing solutions.

Compared to the state-of-the-art maximum k -defective clique algorithm in (Dai et al., [2024a](#)), which achieves a non-trivial time complexity by utilizes a condition to restrict the scenarios of pivoting, directly applying this approach to our pivoting-based algorithm would degrade the time complexity to $O^*(2^n)$. In contrast, we employ a more refined pivoting strategy that considers both the number of non-edges between S and C (lines 10-13 in Algorithm [2](#)) and the number of pivot vertex's non-neighbors (lines 15-18 in Algorithm [2](#)), thereby effectively bounding the time complexity.

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

fewer than k non-neighbors are between S and C , which reduces the size of branching tree and thereby improves the time complexity (see the proof of Theorem 6).

Compared to the maximal biclique algorithm in (Abidi et al., 2020), our pivot vertex selection approach is much different. During the pivoting process, (Abidi et al., 2020) considers the common neighbors, and selects pivot vertex in a heuristic topological order from only one side, which may not be always optimal. In contrast, our pivoting-based branching strategy selects pivot vertex from both side through deterministic condition, which always generates fewest new sub-branches in each recursion.

4. Optimization Techniques

In this section, we propose various optimization techniques, including graph reduction techniques to eliminate redundant vertices, upper-bounding techniques to prune unnecessary instances, and a heuristic approach for identifying an initial large k -defective biclique. These techniques can be easily integrated into our proposed algorithms in Sec. 3, and can significantly enhance their efficiency.

4.1. Graph Reduction Techniques

Common-neighbor-based reduction. For two vertices u and v on the same side, let $cn(u, v)$ (or $cn_S(u, v)$) denote the number of common neighbors of u and v in G (or in S). Given a bipartite graph $G = (U, V, E)$ and a k -MDB $D = (U_D, V_D, E_D)$ of G , every edge $(u, v) \in E_D$ satisfies that (1) $d_D(u) \geq \theta - k$, and (2) for all $w \in N_D(v)$, $cn_D(u, w) \geq \theta - k$. By combining the two conditions, we can conclude that for an edge $(u, v) \in E$, if v has fewer than $\theta - k$ neighbors w such that $cn(u, w) \geq \theta - k$, the edge (u, v) can be pruned in G .

Based on this idea, we present Algorithm 3 to prune such edges from G , referred to as **CnReduction**. The algorithm enumerates all edges (u, v) in ascending order of u 's degree (lines 1-2). For each edges (u, v) , the algorithm counts common neighbors between u and the neighbors w

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why
HTML?

Report
Issue

Back to
Abstract

Download
PDF

LEMMA 0.

Given a bipartite graph $G = (U, V, E)$, the complexity of Algorithm 3 is $O(d \cdot m)$, where $d = \max_{u \in U \cup V} d(u)$.

In practice, Algorithm 3 performs efficiently although its time complexity is $O(d \cdot m)$. This is because in real-world graphs, most vertices have degrees much smaller than d . As a result, the actual runtime of the algorithm often does not reach this worst-case time complexity.

Input: Bipartite graph $G = (U, V, E)$, integers k and θ

Output: A reduced bipartite graph

```

1   $\mathcal{O} \leftarrow$  the ascending degree order of  $U \cup V$ ;
2  for  $u \in \mathcal{O}$  do
3    for  $v \in N(u)$  do
4      for  $w \in N(v)$  do
5         $cn(w) \leftarrow 0$ ;
6
7
8    for  $v \in N(u)$  do
9      for  $w \in N(v)$  do
10         $cn(w) \leftarrow cn(w) + 1$ ;
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
```

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why
HTML?

Report
Issue

Back to
Abstract

Download
PDF

```

1 | return G;
2 |
3 |

```

Algorithm 3 CnReduction(G, k, θ)

One-non-neighbor reduction. For an instance (S, C) and a vertex u , let C_u denote the subset of C where all vertices are on the same side as u . If a branching vertex has only one non-neighbor in $S \cup C$, removing it from C can lead to the pruning of additional vertices. The following lemma explains this idea in detail.

THEOREM 2.

Given an instance (S, C) and a vertex $u \in C$ with $\bar{d}_{S \cup C}(u) = 1$, if u is removed from C , all vertices $v \in C_u$ with $\bar{d}_S(v) \geq 1$ can be pruned from C . Moreover, if $d_C(u) = 1$ and w is the unique non-neighbor of u , then all vertices in $\tilde{N}_C(w)$ can also be pruned from C .

Proof.

Assume D is a k -MDB derived from (S, C) excluding u . For any $v \in C_u$ such that $v \in D$ and $\bar{d}_S(v) \geq 1$, we have $D \setminus \{v\} \cup \{u\}$ is also a k -MDB. In the case where $\bar{d}_C(u) = 1$ and w is the unique non-neighbor of u , if $w \notin D$, then $D \cup \{u\}$ forms a larger k -MDB. Otherwise, for any $v \in \tilde{N}_C(w)$ such that $v \in D$, we have $D \setminus \{v\} \cup \{u\}$ is also a k -MDB. ■

Ordering-based reduction. Given a bipartite graph $G = (U, V, E)$, let $u_1, u_2, \dots, u_{|U|}$ be an arbitrary vertex order of U . For each vertex u_i , the ordering-based reduction aims to find a k -MDB D_i including u_i while excluding $u_1, u_2, \dots, u_{i-1}$. Obviously, the k -MDB of G is the one with most edges among $D_1, D_2, \dots, D_{|U|}$. Next, we extend Lemma 2 to constrain the vertex distance in D_i , as shown in the following lemma.

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

Lemma 3 indicates that when searching for D_i , vertices with distances more than 3 from u_i can be pruned. Let $N^2(u) = \{w \in N(v) | v \in N(u)\}$, i.e., the set of u 's 2-hop neighbors. Let $N_+^2(u_i) = N^2(u_i) \cap \{u_{i+1}, u_{i+2}, u_{|U|}\}$. According to Lemma 3, we can construct an initial instance (S_i, C_i) to find D_i , where $S_i = (\{u_i\}, \emptyset)$ and $C_i = (N_+^2(u_i), \bigcup_{u_j \in N_+^2(u_i)} N(u_j))$.

In practical implementation, to identify a larger k -defective biclique as early as possible, we traverse the vertices in U in descending order of their degrees. This approach is based on the intuition that vertices with higher degrees are more likely to be present in an MDB. Additionally, the common-neighbor-based reduction techniques can be employed on (S_i, C_i) to prune unnecessary vertices and edges. Specifically, vertices $u \in C_i$ are pruned if $d_{S_i \cup C_i}(u) < \theta - k$, and edges (u, v) are pruned if there are fewer than $\theta - k$ neighbors w of v satisfying $cn(u, w) \geq \theta - k$.

Progressive bounding reduction. The size threshold θ is provided as an input parameter and can be set to a very small value (e.g., $= k + 1$), which may limit the effectiveness of our optimization techniques. To address this issue, we utilize the progressive bounding technique inspired from the maximum biclique problem (Lyu et al., 2020) to provide tighter size thresholds for the k -MDB search problem.

Specifically, let θ_U^t and θ_V^t denote the size thresholds in t -th round. The objective of t -th round is to identify a k -MDB $D_t = (U_{D_t}, V_{D_t}, E_{D_t})$ satisfying $|U_{D_t}| \geq \theta_U^t$ and $|V_{D_t}| \geq \theta_V^t$. Let $D^* = \max\{D_1, D_2, \dots, D_{t-1}\}$ be the currently largest k -defective biclique. The progressive bounding reduction generates θ_U^t and θ_V^t in the following way:

$$(3) \quad \theta_U^t = \max\{\theta, \lfloor \theta_U^{t-1} / 2 \rfloor\},$$

$$(4) \quad \theta_V^t = \max\{\theta, \lfloor |E_{D^*}| / \theta_U^{t-1} \rfloor\}.$$

Specially, $\theta_U^0 = \max_{u \in U} d_U(u) + k$, and the iteration terminates when $\theta_U^t = 0$. The following lemma

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

Given a bipartite graph G and a k -MDB $D = (U_D, V_D, E_D)$ of G , there always exists an integer t such that $\theta_U^t \leq |U_D|$ and $\theta_V^t \leq |V_D|$.

In many real-world graphs, θ_U^t and θ_V^t are often significantly larger than θ , which helps facilitate more effective graph reduction, such as a smaller reduced graph by performing the common-neighbor-based reduction.

Discussion. Several prior studies have proposed similar graph reduction techniques for other subgraph models (Dai et al., [2024a](#); Yu and Long, [2023](#); Dai et al., [2024b](#); Wang et al., [2023](#); Zhou et al., [2025](#)). The progressive bounding reduction have been widely applied on these models (Yu and Long, [2023](#); Wang et al., [2023](#); Zhou et al., [2025](#)). In this work, we adapted these techniques to our problem by incorporating the unique properties of k -defective bicliques. However, our proposed common-neighbor-based reduction and one-non-neighbor reduction are technically distinct from all prior approaches.

On the one hand, the common-neighbor-based reduction leverages the constraints on vertex degree and common neighbors in a k -MDB. The reduction based on vertex degree has been widely applied in the form of core (Yu and Long, [2023](#); Zhou et al., [2025](#)). However, this approach results in a very limited reduction in graph size. Additionally, by applying a single constraint on common neighbors of two vertices u and v where $cn(u, v) < \theta - k$, we can conclude that u and v cannot co-exist in the same k -MDB. However, maintaining such coexistence relationships requires $O(n^2)$ space, making it quite challenging for practical implementation.

On the other hand, the one-non-neighbor reduction focuses on analyzing vertices in C that have only one non-neighbor in the current subgraph $G(S \cup C)$. A relevant prior technique addresses the maximum k -defective clique search problem (Dai et al., [2024a](#)), where a vertex in C with exactly one non-neighbor in the subgraph can be directly added to S . However, this reduction cannot be directly applied to the k -MDB problem, as such a vertex may not belong to any k -MDB. For example, consider the following 1-MDB search instance (S, C) , where $|U_S| = 2$, $|V_S| = 3$, $U_C = \{u\}$ and $V_C = \{v\}$. $G(S)$ has one non-edge, u and v are non-neighbors. Directly adding v to S results in a non-

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why
HTML?

Report
Issue

Back to
Abstract

Download
PDF

4.2. Novel Upper-Bounding Techniques

In this subsection, we propose a novel upper-bounding technique that provides effective upper bounds for both vertices and edges. Formally, given an instance (S, C) , assume $u_1, u_2, \dots, u_{|U_C|}$ and $v_1, v_2, \dots, v_{|V_C|}$ are the vertices of U_C and V_C in ascending order of $\bar{d}_S(\cdot)$, respectively. Let $\bar{d}_S^i(U_C) = \sum_{t=1}^i \bar{d}_S(u_t)$ and $\bar{d}_S^j(V_C) = \sum_{t=1}^j \bar{d}_S(v_t)$, i.e., the total number of non-neighbors in S among the first i vertices of U_C and V_C . Based on this, the vertex upper bound is presented in the following lemma.

LEMMA 0 (Vertex upper bounds).

For any k -MDB $D = (U_D, V_D, E_D)$ derived from (S, C) , we have $|U_D| \leq |U_S| + i$ and $|V_D| \leq |V_S| + j$, where i and j are the largest values satisfying $\bar{d}_S^i(U_C) \leq k - |\bar{E}_S|$ and $\bar{d}_S^j(V_C) \leq k - |\bar{E}_S|$, respectively.

Proof.

For the side of U , it is obvious that i is the maximum number of vertices that can be added from U_C to U_S such that $(U_S \cup \{u_1, u_2, \dots, u_i\}, V_S)$ remains a valid k -defective biclique. For any k -MDB derived from (S, C) consisting of $(U_S \cup U_{C'}, V_S \cup V_{C'})$, $(U_S \cup U_{C'}, V_S)$ is still a k -defective biclique. Therefore, we have that $|U_{C'}| \leq i$. The proof on the other side is similar. ■

Based on Lemma 5, if both sides reach their vertex upper bounds, an edge upper bound can be calculated as $(|U_S| + i) \times (|V_S| + j) - |\bar{E}_S| - \bar{d}_S^i(U_C) - \bar{d}_S^j(V_C)$. However, this upper bound does not account for the coexistence of vertices in U_C and V_C . To solve the issue, we present a tighter edge upper bound in the following lemma.

LEMMA 0 (Edge upper bound).

For any k -MDB D derived from (S, C) , we have that $|E_D| \leq \max_{0 \leq i \leq |U_C|} (|U_S| + i) \times (|V_S| + j) - |\bar{E}_S| - \bar{d}_S^i(U_C) - \bar{d}_S^j(V_C)$, where j is the largest value satis-

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

Consider a new instance (S, C') reconstructed from (S, C) by fully connecting vertices between $U_{C'}$ and $V_{C'}$, i.e., $\bar{E}_C = \emptyset$. The ordering of vertices in C' with respected to $\bar{d}_S(\cdot)$ remains unchanged. There exists a k -MDB derived from (S, C') composed of S , a prefix of $\{u_1, u_2, \dots, u_{|U_C|}\}$, and a prefix of $\{v_1, v_2, \dots, v_{|V_C|}\}$ (otherwise, for any vertex not in a prefix, a preceding vertex can always be found to replace it). Moreover, the k -MDB derived from (S, C') have more edges than any k -MDB derived from (S, C) . Consequently, Lemma 6 provide a correct edge upper bound. ■

```

1 |
2 |
Input: Instance  $(s = (U_S, V_S), c = (U_C, V_C))$  and integer  $k$ 
Output: two vertex upper bounds and an edge upper bound
3 |
4 |  $k' \leftarrow k - |\bar{E}_S|;$ 
5 | Let  $u_1, u_2, \dots, u_{|U_C|}$  and  $v_1, v_2, \dots, v_{|V_C|}$  be the vertices of  $U_C$  and  $V_C$  in ascending order of  $\bar{d}_S(\cdot)$ ;
6 |  $\bar{d}_U \leftarrow 0; \bar{d}_V \leftarrow 0; c_U \leftarrow 0; c_V \leftarrow 0;$ 
7 | for  $i \leftarrow 1 \dots |U_C|$  s.t.  $\bar{d}_U + \bar{d}_S(u_i) \leq k'$  do
8 |    $\bar{d}_U \leftarrow \bar{d}_U + \bar{d}_S(u_i);$ 
9 |    $c_U \leftarrow c_U + 1;$ 
10 |
11 |
12 |  $e \leftarrow (|U_S| + c_U) \cdot |V_S| - |\bar{E}_S| - \bar{d}_U;$ 
13 |
14 |  $i \leftarrow c_U;$ 
15 |
16 | for  $j \leftarrow 1 \dots |V_C|$  s.t.  $\bar{d}_V + \bar{d}_S(v_j) \leq k'$  do
17 |
18 |   while  $\bar{d}_U + \bar{d}_V + j > k'$  do
19 |
20 |      $\bar{d}_U \leftarrow \bar{d}_U - \bar{d}_S(u_i);$ 
21 |
22 |      $i \leftarrow i - 1;$ 
23 |
24 |
25 |    $\bar{d}_V \leftarrow \bar{d}_V + \bar{d}_S(v_j);$ 
26 |
27 |    $c_V \leftarrow c_V + 1;$ 
28 |
29 |    $e \leftarrow \max\{e, (|U_S| + i) \cdot (|V_S| + j) - |\bar{E}_S| - \bar{d}_U - \bar{d}_V\};$ 
30 |
31 |

```

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why
HTML?

Report
Issue

Back to
Abstract

Download
PDF

Algorithm 4 UpperBound $((S, C), k)$

Implementation details 0.

Algorithm 4 presents a pseudocode of our upper-bounding techniques. It starts by setting k' as the remaining number of missing edges (line 1), and sorting the vertices of U_C and V_C in ascending order of $\tilde{d}_S(\cdot)$ (line 2), which can be efficiently done using a bucket sort in linear time and space. Then, the algorithm calculates the vertex upper bound in U by finding the longest prefix of U_C that can be added to U_S (lines 4-6), where c_U records the prefix length. After that, the algorithm derives the upper bound c_V for vertices in V using the same way, and simultaneously calculates the upper bound e for edges by removing vertices from the end of the prefix of U_C while computing the longest prefix of V_C (lines 9-15). Finally, it returns two upper bounds of both sides as well as an edge upper bound. We present the time complexity of the algorithm in the following lemma.

LEMMA 0.

The time complexity of Algorithm 4 is $O(n)$.

4.3. A Heuristic Approach

We propose a heuristic approach to efficiently calculate a large k -defective biclique. We observe that in many real-world graphs, one side of the k -MDB with size threshold θ contains exactly θ vertices, while the other side contains much more than θ vertices. Motivated by this, our heuristic approach aims to find a large k -defective biclique with one side has θ vertices. Specifically, given a graph $G = (U, V, E)$ and an integer k , we initially set two vertex subset $U' = \emptyset$ and $V' = V$. let $u \in U$ be the vertex with largest $d_{V'}(u)$. If $G(U' \cup \{u\}, V')$ has more than k non-edges, we iteratively delete vertices $v \in V$ with largest $\tilde{d}_{U' \cup \{u\}}(v)$ until $G(U' \cup \{u\}, V')$ has no more than k non-edges, and then we add u to U' . We repeat this process until $|U'| = \theta$, and yield a large k -MDB $G(U', V')$ if $|V'| = \theta$. Otherwise, we yield an empty answer. It is easy to verify that this approach can be finished in $O(\theta n + m)$, which is linear as θ is always a small value.

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

the algorithm (as shown in Fig. 4b and Fig. 4c). Specifically, for a given branching strategy, once an instance (S, C) corresponding to a branch is determined, the sub-branches generated by this branch are uniquely defined and mutually independent. Therefore, we parallelize the processing of sub-branches at lower levels of the search tree. Additionally, the parallelization process is controlled by a global counter that tracks the total number of parallel tasks, and when the counter reaches a predefined threshold, the sub-branches generated by each branch are processed sequentially. Additionally, we also perform parallelization on the ordering-based reduction through the traversal of ordered vertices, and on the common-neighbor-based reduction through the loop in lines 2-13 of Algorithm 3.

Input: A bipartite graph $G = (U, V, E)$, integers k, θ

Output: A k -MDB of G

```

1   $D^* \leftarrow \text{Heuristic}(G, k, \theta);$ 
2   $\theta_U \leftarrow \max_{u \in U} d(u);$ 
3  while  $\theta_U > \theta$  do
4     $\theta_V \leftarrow \max\{\theta, \lfloor |D^*| / \theta_U \rfloor\}; \theta_U \leftarrow \max\{\theta, \lfloor \theta_U / 2 \rfloor\};$ 
5     $G' \leftarrow \text{parallel CnReduction}(G, k, \min\{\theta_U, \theta_V\});$ 
6     $\{u_1, u_2, \dots, u_{|U|}\} \leftarrow \text{the descending degree order of } U;$ 
7    parallel for  $i \in 1 \dots |U|$  do
8       $S \leftarrow (\{u_i\}, \emptyset); C \leftarrow (N_+^2(u_i), \bigcup_{v \in N_+^2(u_i)} N(v));$ 
9       $C \leftarrow \text{Reduce}(C, u_i, \theta_U, \theta_V);$ 
      BranchP $^*(S, C);$  // BranchP equipped with graph reduction techniques, upper-bounding techniques and
parallelization
1  |
0  |
1  |
1  |
1  |
2  return  $D^*$ ;
1  |
3  |
1  |
4  Function  $\text{Reduce}(C = (U_C, V_C), u, \theta_U, \theta_V)$ :
1  |
5     $U_{C'} \leftarrow \{v \in U_C | \text{cn}(u, v) \geq \theta_V - k\};$ 
1  |
6     $V_{C'} \leftarrow \{v \in V_C | d(v) \geq \theta_U - k\};$ 
1  |
7    return  $(U_{C'}, V_{C'})$ 
1  |
8  |
```

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

All the proposed optimization techniques can be applied to our branch-and-bound algorithm and pivoting-based algorithm. We refer to the branch-and-bound algorithm and pivoting-based algorithm equipped with all the optimization techniques as MDBB and MDBP, respectively. An implementation of MDBP is presented in Algorithm 5. The algorithm begins by obtaining an initial k -defective biclique (line 1) by performing the heuristic approach. Then it starts to find k -MDB by performing the progressive bounding reduction (lines 2-4). In each iteration, two size thresholds θ_U and θ_V are generated based on Eq. (3) and Eq. (4). Then, the parallelized common-neighbor-based reduction are employed on the input graph G (line 5) by invoking CnReduction (Algorithm 3). the algorithm continues by performing the parallelized ordering-based reduction on the reduced graph (lines 6-10). In detail, it traverses each vertex u of U in descending degree order (lines 7-8), and constructs an initial instance (S, C) . After that, the algorithm further reduce the size of C by invoking Reduce (line 9), which utilizes the common-neighbor-based reduction (lines 13-14). Finally, the algorithm runs BranchP* to find k -MDB in (S, C) , which is adapted from BranchP in Algorithm 2 by additionally implementing the graph reduction techniques and the upper-bounding techniques (Algorithm 4). By taking the maximum returned solution from all invocations of BranchP*, the algorithm derives a k -MDB D^* at the end and returns D^* as the final answer (line 11). The implementation of MDBB is quite similar to that of MDBP, which differs only in that replacing BranchP* in Algorithm 5 with BranchB from Algorithm 1 incorporating the optimization techniques used in BranchP*.

5. Experiments

5.1. Experimental Setup

Datasets. We collect 11 real-world bipartite graphs from diverse categories to evaluate the efficiency of our algorithms. These graphs have been widely used as benchmark datasets for evaluating the performance of other bipartite subgraph search problem (Lyu et al., 2020; Yu and Long, 2023; Dai et al., 2023b). The detailed information of these graphs are shown in Table 1, where Δ and $\bar{d}$ represent the maximum degree and average degree of vertices on each side, respectively. All the datasets are available at <http://konect.cc/networks>.

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why
HTML?
Report
Issue
Back to
Abstract
Download
PDF

branching strategy at line 9-10 in Algorithm 1 with the straightforward strategy that randomly selecting a branching vertex u from C . On the other hand, MDCadp is adapted from the state-of-the-art maximal defective clique algorithm Pivot+ (Dai et al., 2023a). In detail, MDCadp converts the input bipartite graph into a traditional graph by treating all vertices on the same side as fully connected and runs Pivot+ on the converted graph, subsequently extracting a k -MDB from all the maximal defective cliques of the converted graph. Furthermore, both MDBase and MDCadp incorporate the optimization techniques outlined in Sec. 4.1 and 4.3. All algorithms are implemented in C++ and executed on a PC equipped with a 2.2GHz CPU and 128GB RAM running Linux.

Parameter settings. During the evaluations of all algorithms, we vary the value of k from 1 to 6. As outlined in Sec. 2, we focus on finding a connected k -MDB, achieved by setting $\theta > k$. Based on this, we vary the value of θ from $k + 1$ to 10 for a specified k .

Table 1. Datasets used in experiments

Dataset	$ U $	$ V $	$ E $	Δ	$\text{mdbk}\bar{d}$
LKML	42,045	337,509	599,858	(31,719, 6,627)	(14, 1)
Team	901,130	34,461	1,366,466	(17, 2,671)	(1, 39)
Mummun	175,214	530,418	1,890,661	(968, 19805)	(10, 3)
Citeu	153,277	731,769	2,338,554	(189,292, 1,264)	(15, 3)
IMDB	303,617	896,302	3,782,463	(1,334, 1,590)	(12, 4)
Enwiki	18,038	2,190,091	4,129,231	(324,254, 1,127)	(228, 1)
Amazon	2,146,057	1,230,915	5,743,258	(12,180, 3,096)	(2, 4)
Twitter	244,537	9,129,669	10,214,177	(640, 18,874)	(41, 1)
Aol	4,811,647	1,632,788	10,741,953	(100,629, 84,530)	(2, 6)
Google	5,998,790	4,443,631	20,592,962	(423, 95,165)	(3, 4)
LiveJournal	3,201,203	7,489,073	112,307,385	(300, 1,053,676)	(35, 14)

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why
HTML?

Report
Issue

Back to
Abstract

Download
PDF

LKML	1	5	84	1.61	2.15	2.36	28.19	2	6	70	7.60	30.14	357.33	-	3	7
	4	7	66	103.03	1,216.70	-	-	5	8	67	107.31	1,224.86	-	-	6	9
		8	60	9.36	46.38	2,360.74	-		9	0	13.42	51.55	3,441.79	-		10
Team	1	2	701	19.26	39.98	43.67	-	2	3	265	45.51	-	-	-	3	4
		5	99	7.14	7.19	7.73	66.18		6	64	9.53	14.09	31.43	-		7
	4	5	111	591.71	-	-	-	5	6	79	2,684.57	-	-	-	6	8
		8	0	9.92	15.89	1,734.29	-		9	0	10.04	16.35	9,791.32	-		10
Mummun	1	2	10,315	18.49	18.62	79.28	-	2	3	8,803	22.88	8,661.39	-	-	3	4
		5	2,469	13.57	13.59	16.49	-		6	1,768	21.08	37.83	84.27	-		7
	4	5	2,481	45.73	-	-	-	5	6	1,783	69.39	-	-	-	6	7
		8	878	38.18	5,538.54	10,440.17	-		9	886	40.64	6,737.14	-	-		10
Citeu	1	2	12,425	141.21	146.81	-	-	2	3	12,427	148.38	187.69	-	-	3	4
		5	7,394	5.48	5.69	-	-		6	4,960	7.88	77.20	-	-		7
	4	5	7,406	141.78	-	-	-	5	6	4,975	158.43	-	-	-	6	7
		8	2,600	25.92	-	-	-		9	2,600	36.03	-	-	-		10
IMDB	1	2	775	5.99	5.39	4,460.09	-	2	3	562	8.86	16.96	-	-	3	4
		5	514	6.12	5.35	145.10	23.43		6	514	6.76	6.91	-	1,123.22		7
	4	5	526	19.99	593.86	-	-	5	6	529	31.05	683.75	-	-	6	7
		8	436	15.13	67.00	-	-		9	436	19.34	170.90	-	-		10
Enwiki	1	2	203,569	3,630.07	-	-	-	2	3	133,759	4,977.99	-	-	-	3	4
		5	21,939	156.70	158.87	354.17	387.15		6	4,240	199.76	582.63	-	-		7
	4	5	21,951	3,826.16	-	-	-	5	6	4,255	5,744.46	-	-	-	6	8
		8	2,316	339.60	-	-	-		9	1,696	394.81	-	-	-		10
Amazon	1	2	1,319	42.68	45.42	-	-	2	3	427	56.54	646.44	-	-	3	4
		5	413	11.71	11.12	12.21	49.70		6	418	12.91	12.00	167.48	-		7
	4	5	428	99.80	850.49	-	-	5	6	433	249.97	-	-	-	6	7
		8	368	13.68	15.96	171.44	-		9	379	16.09	20.38	319.93	-		10
Twitter	1	2	5,369	11.13	13.24	46.13	-	2	3	5,383	10.88	51.42	2,143.04	-	3	4
		5	5,369	6.27	7.60	36.58	8,976.29		6	5,383	6.63	19.51	1,432.31	-		7
	4	5	5,411	20.75	3,549.73	-	-	5	6	5,425	31.20	-	-	-	6	7

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why
HTML?

Report
Issue

Back to
Abstract

Download
PDF

		θ	$ E^* $	Time (s)	Speedup			θ	$ E^* $	Time (s)	Speedup			θ
Google	1	2	10,829	2,163.49	2,525.77	4,814.91	-	2	3	3,370	4,121.31	-	-	-
		5	439	552.42	573.02	629.33	-		6	184	898.04	2,771.57	4,077.91	-
	4	7	157	5,331.22	-	-	-	5	8	155	6,750.40	-	-	-
		9	140	613.17	9,387.13	-	-		10	145	683.84	-	-	-

5.2. Performance Studies

Exp1: Efficiency comparison among different algorithms. We evaluate the efficiency of two proposed algorithms MDBB and MDBP, along with two baseline algorithms MDBase and MDCadp. The results are shown in Table 2, where the column $|E^*|$ represents the number of edges in the k -MDB, and the notion “-” represents a timeout exceeding 3 hours. As observed, MDBP shows superior performance across most input parameters, achieving two order of magnitude faster than MDCadp on most datasets, and two order of magnitude faster than MDBase on most datasets when $k > 3$. Notably, MDBP essentially solves all tests on most of datasets when setting $k \geq 3$, while MDBase and MDCadp can hardly finish any tests within the time limit. For example, on the *Mummun* dataset with $k = 3$ and $\theta = 7$, MDBase takes 3,677 seconds and MDCadp times out at 108,000 seconds, whereas MDBP completes in just 34 seconds, yielding speedups of $100 \times$ over MDBase and $1000 \times$ over MDCadp. On the other hand, when $k \leq 2$ and $\theta \geq 5$, the efficiency of MDBP is comparable to MDBB. However, for $k \geq 3$ or $\theta \leq 4$, MDBP demonstrates a significant advantage. For example, on the *Twitter* dataset with $k = 1$ and $\theta = 5$, both MDBP and MDBB runs for about 7 seconds, but with $k = 4$ and $\theta = 5$, MDBB takes about 1 hour while MDBP completes in just 20 seconds, achieving a speedup of more than $150 \times$. These results underscore the effectiveness and high efficiency of the pivoting-based branching strategy mentioned in Sec. 3.2. Additionally, MDBB retains advantages over MDBase and MDCadp, highlighting the effectiveness of the new binary branching strategy in Sec. 3.1.

Exp2: Runtime with varying θ and k . We evaluate the performance of 4 algorithms MDBB, MDBP, MDBase and MDCadp with varying θ and k on two representative datasets, *IMDB* and *Google*. The results are shown in Fig. 4, where the notion “INF” represents the runtime exceeds 3 hours. As observed, both MDBP and MDBB consistently outperform baseline algorithms MDBase and MDCadp

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

to input parameters and superior scalability. Additionally, we evaluate the performance of MDBB and MDBP in searching for the maximum biclique by setting $k = 0$. We compare them with a state-of-the-art maximum biclique search algorithm MBC (Lyu et al., 2020). The results are shown in Fig. 4e and Fig. 4f. As observed, MDBP and MDBB exhibit an advantage over MBC due to two main factors: (1) both of our algorithms achieves better time complexities than MBC, which has a time complexity of $O^*(2^n)$; and (2) our algorithms incorporate more advanced heuristic approach and graph reduction techniques, further enhancing their efficiency.

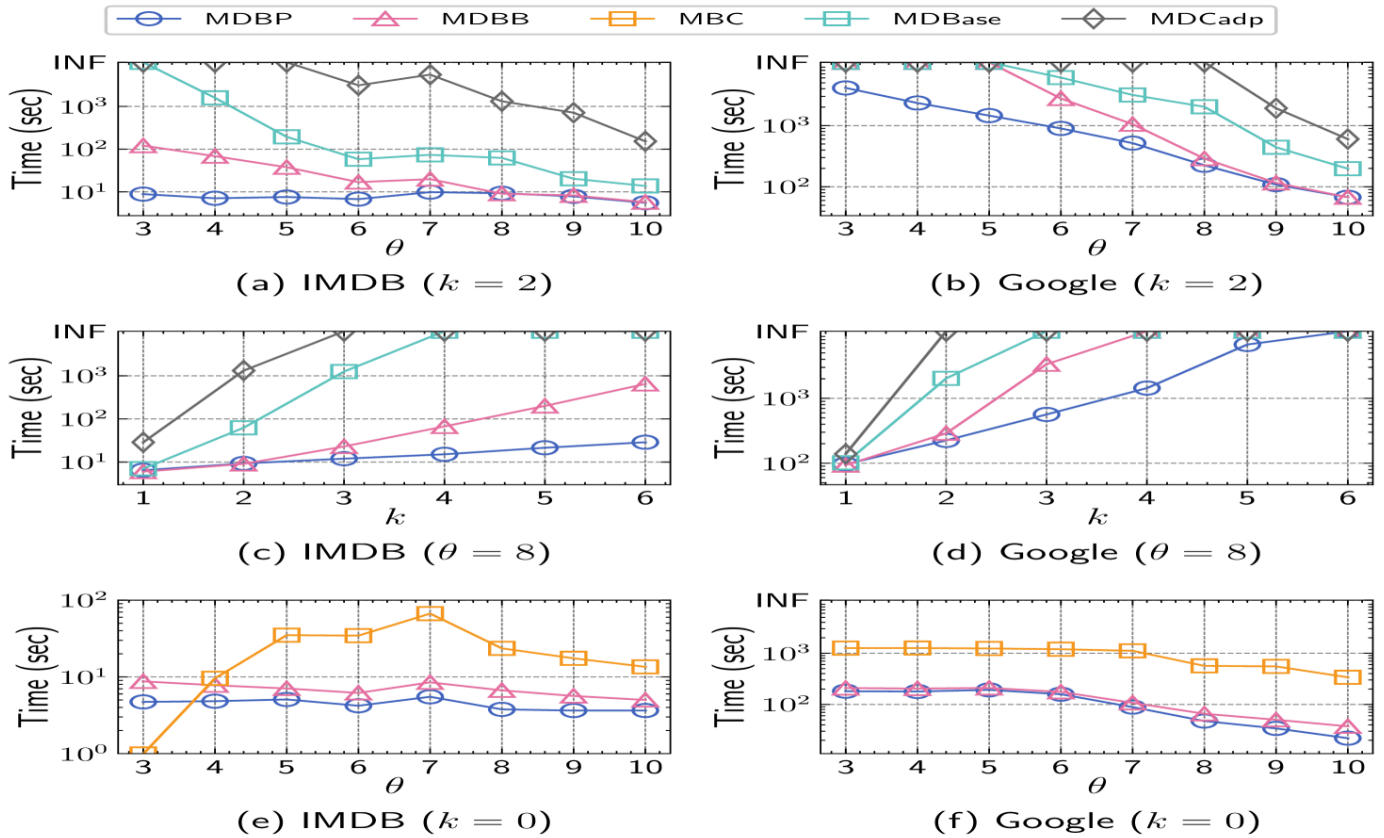


Figure 4. Runtime with varying θ and k

This is **experimental HTML** to improve accessibility. We invite you to report rendering errors. Learn more about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

tiveness of our upper-bounding techniques. Additionally, we observe that MDBP-H still always performs better than MDBB when $\theta \leq 6$ or $k \geq 3$, which further highlights the high efficiency of our pivoting techniques.

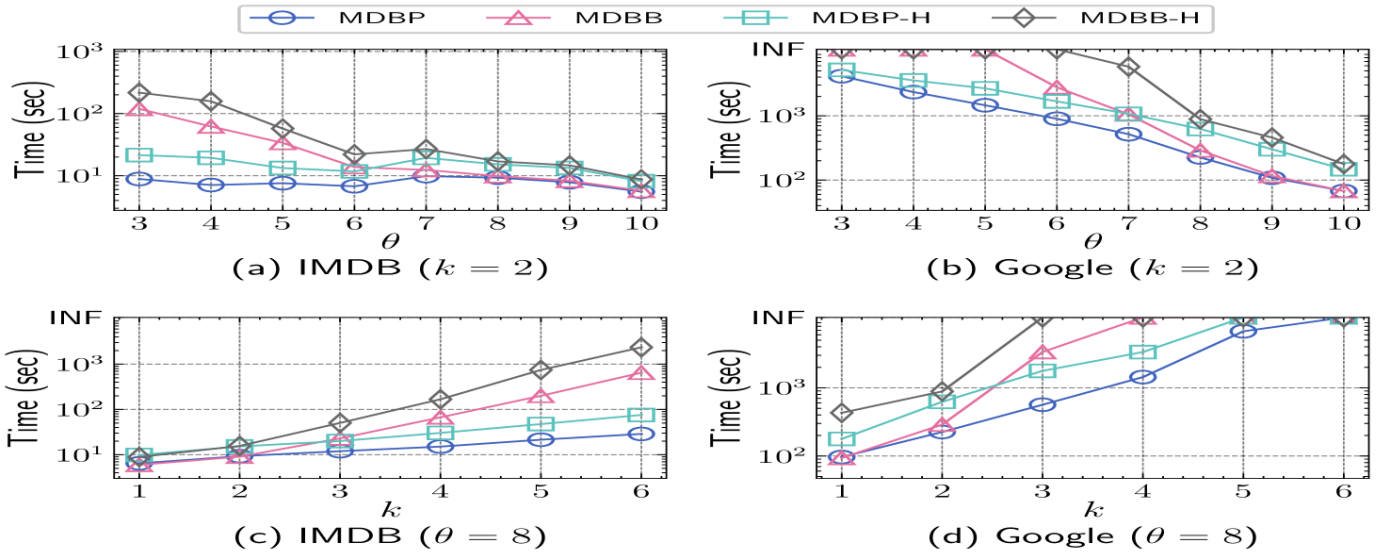


Figure 5. Runtime without using the heuristic approach

Exp4: Efficacy of graph reduction techniques. To evaluate the effectiveness of our graph reduction techniques, we implement 4 ablation versions of MDBP and MDBB by successively removing the following reductions: one-non-neighbor reduction, common-neighbor-based reduction, progressive bounding reduction and ordering-based reduction. The results are presented in Fig 6, which label the original algorithm as “Original”, and label the 4 ablation versions as “-1”, “-1C”, “-1CP”, “-1CPO”. As observed, all graph reduction techniques contribute to efficiency improvements. Specifically, the one-non-neighbor and common-neighbor-based reductions yield stable performance improvements with varying θ and k , while the progressive bounding and ordering-based reduction demonstrate more pronounced improvements when θ decreases or k increases. Notably, when $k = 1$ or $\theta = 10$, versions of MDBB and MDBP without progressive bounding reduction still out-

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

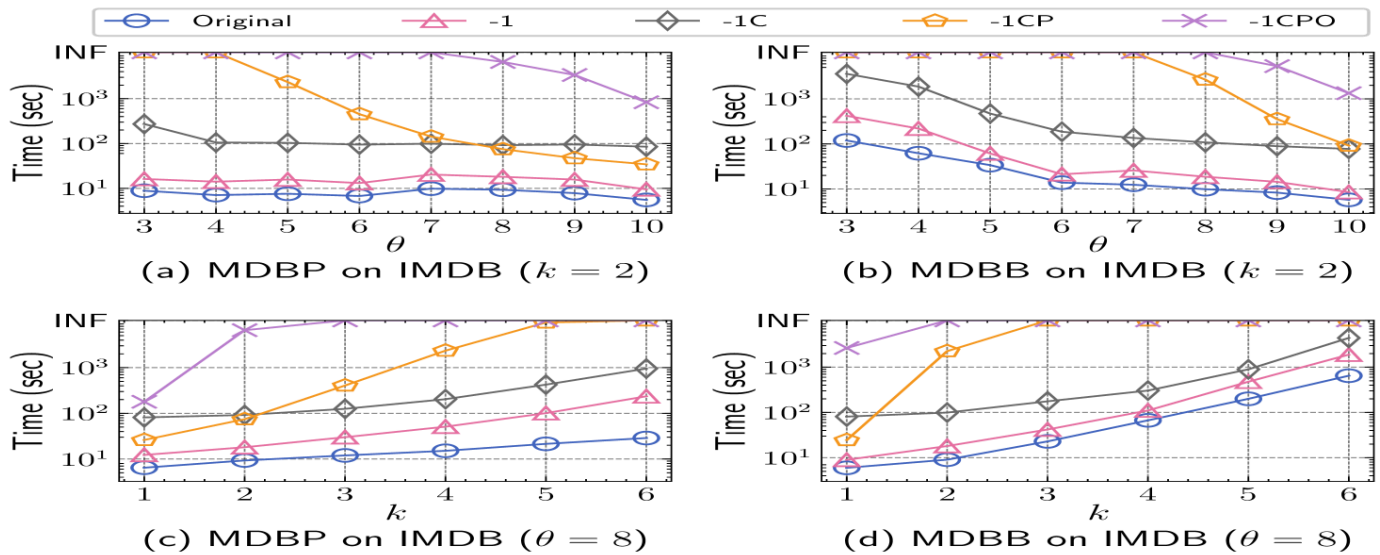


Figure 6. Runtime without graph reduction techniques

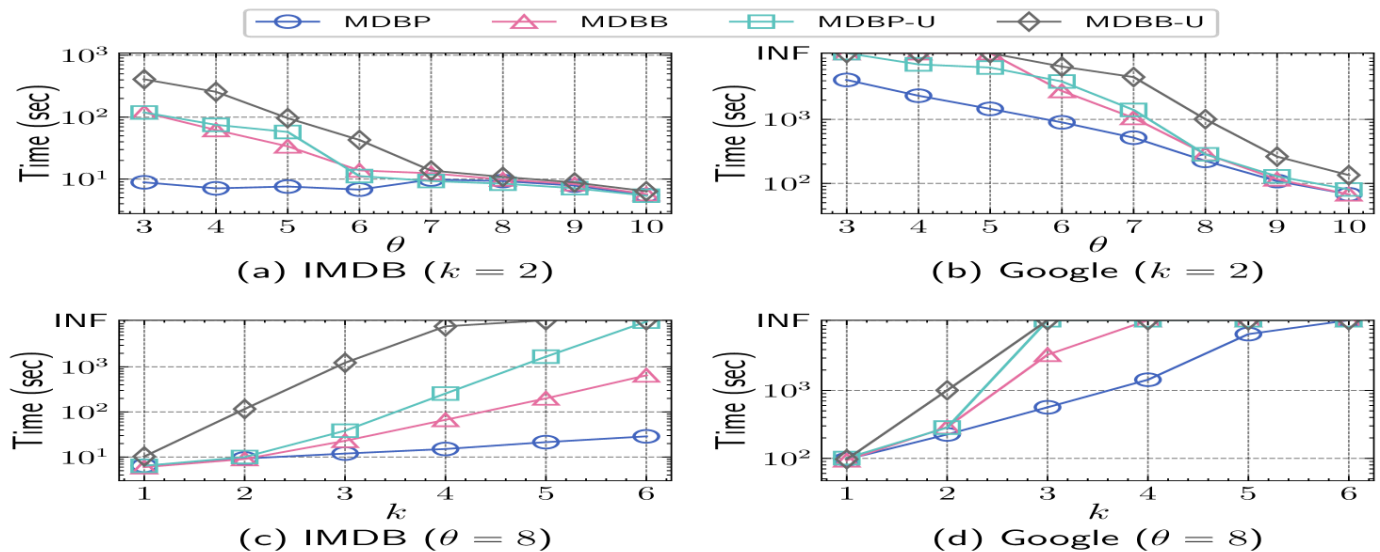


Figure 7. Runtime without upper-bounding techniques

increases or θ decreases, indicating the strong effectiveness and efficiency of our proposed upper-bounding techniques. Additionally, the improvement of MDBP by performing upper-bounding techniques is more substantial compared to MDBB. This is likely because the pivoting-based branching strategy introduces more non-edges to the partial set S earlier, resulting in tighter vertex and edge upper bounds derived from the upper-bounding techniques, which helps prune more unnecessary instances.

Table 3. Runtime on synthetic graphs (in seconds)

Distribution	ρ	Δ	$\bar{d}$	$k = 1, \theta = 2$			$k = 3, \theta = 4$			$k = 5, \theta = 6$		
				$ E^* $	MDBP	MDBB	$ E^* $	MDBP	MDBB	$ E^* $	MDBP	MDBB
Power-law	0.2	(97, 82)	(19, 19)	197	0.10	0.18	181	0.29	0.96	175	8.03	11.82
	0.3	(77, 99)	(29, 29)	269	0.11	0.33	277	0.56	9.52	292	16.02	133.29
	0.4	(95, 100)	(36, 36)	349	0.27	0.51	357	1.76	26.01	375	33.84	432.58
	0.5	(100, 99)	(45, 45)	482	0.92	1.94	494	5.83	105.49	506	189.92	2,030.16
Normal	0.2	(29, 29)	(17, 17)	27	0.26	0.47	29	31.74	75.84	32	59.74	153.87
	0.3	(43, 43)	(29, 29)	44	1.24	3.04	45	179.49	397.23	49	312.55	1,225.26
	0.4	(48, 51)	(39, 39)	56	3.78	10.73	67	489.87	1,015.43	65	1,300.01	-
	0.5	(62, 61)	(49, 49)	65	15.03	43.81	65	2,095.58	-	-	-	-

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

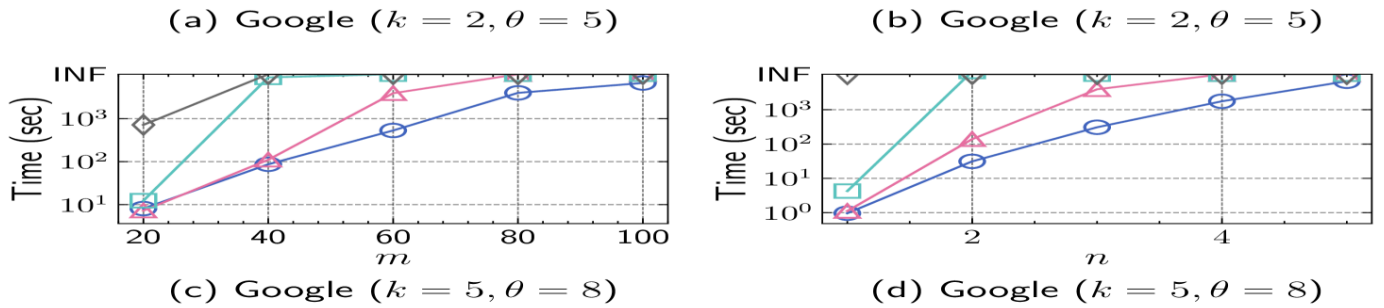


Figure 8. Scalability of different algorithms

Exp6: Performance on synthetic graphs. To discover the relationship between the efficiency of our approaches and graph characteristics, we evaluate the runtime of MDBB and MDBP on synthetic graphs with varying vertex degree distributions and graph densities. Specifically, we generate 8 synthetic graphs with 100 vertices on each side. Their vertex degrees follow a power-law or normal distribution, and the density $\rho = \frac{|E|}{|U| \cdot |V|}$ varies from 0.2 to 0.5. The results are presented in Table 3. As observed, our algorithms consistently exhibit high efficiency on power-law distribution graphs, whereas their performance on normal distribution graphs is more significantly influenced by ρ and k . This phenomenon can be attributed to two key factors. First, Power-law graphs exhibit a larger gap between Δ and $\bar{d}$ than normal graphs, which facilitates our branching strategies to eliminate more redundant branches and enables our graph reduction techniques to exclude more vertices. This explains why our approaches are highly efficient on large-scale real-world graphs with a significantly gap between Δ and $\bar{d}$, as illustrated in Table 1 and 2. Second, the higher number of k -MDB edges in power-law graphs benefits our upper bound techniques for effectively pruning suboptimal branches, which can also be reflected in the real-world graphs in Table 2. Additionally, We observe that MDBP outperforms MDBB on these synthetic graphs, indicating the effectiveness and advancement of our pivoting-based branching strategy.

Exp7: Scalability testing. To evaluate the scalability of algorithms, we test the runtime of MDBB, MDBP, MDBase and MDCadp on the Google dataset using random sampling of vertices or edges. Results on the other datasets are consistent. We prepare 8 proportionally sampled subgraphs of Google by sampling 20%, 40%, 60% and 80% vertices or edges from the original graph. The results

This is **experimental HTML** to improve accessibility. We invite you to report rendering errors. Learn more about this project and help improve conversions.

Why
HTML?

Report
Issue

Back to
Abstract

Download
PDF

timization techniques.

Table 4. Runtime of parallelized algorithms (in seconds)

Dataset	k	θ	MDBP with threads			MDBB with threads		
			1	10	20	1	10	20
Google	1	2	2,163.49	425.76	34.69	3,052.03	870.92	305.05
	3	4	6,250.09	1,324.98	802.67	-	-	10,649.47
	5	6	-	-	8,947.28	-	-	-
LiveJournal	1	2	119.61	89.14	47.11	-	-	9,314.33
	3	4	3,996.77	997.90	850.52	-	-	-
	5	6	-	-	10,016.72	-	-	-

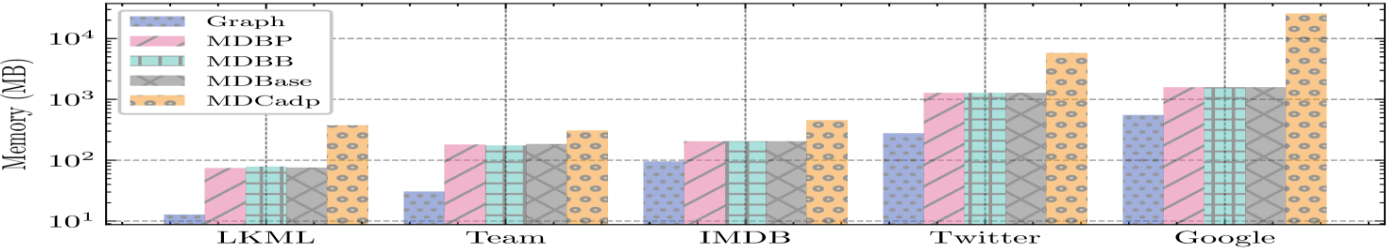


Figure 9. Memory usage of various algorithms

Exp8: Efficiency of parallelized algorithms. To evaluate the efficiency of the parallel version of $\blacktriangle$ MDBB and MDBP, we test their runtime on two large-scale graphs, *Google* and *LiveJournal*, which are two hard cases for our sequential algorithms as shown in Table 1. The results are shown in Table

This is **experimental HTML** to improve accessibility. We invite you to report rendering errors. Learn more about this project and help improve conversions.

Why
HTML?

Report
Issue

Back to
Abstract

Download
PDF

Exp9: Memory usage. In this experiment, we evaluate the memory usage of algorithms MDBB, MDBP, MDBase and MDCadp on five representative graphs. We measure the maximum physical memory (in MB) used by these algorithms with the parameters $k = 2$ and $\theta = 3$. Similar results can also be observed with other parameters. The results are shown in Fig. 9. As observed, all algorithms except MDCadp exhibit comparable space consumption, which is approximately proportional to the size of input graph. Notably, the space consumed by MDCadp is higher than other algorithms. This is mainly because MDCadp requires to fully connect vertices on both sides of a bipartite graph, resulting in increased graph density and a larger search space. These results indicate that our proposed algorithms are space efficient.

5.3. Case Study

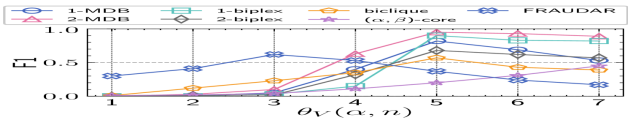
We conduct a case study to evaluate the effectiveness of k -MDB in a fraud detection task, following the experimental setup described in (Hooi et al., 2016). We simulate a camouflage attack scenario on the dataset *Appliance* (<https://amazon-reviews-2023.github.io/>), which contains 602,777 reviews on 30,459 products by 515,650 users. We inject a fraud block consisting of 500 fake users, 500 fake products, along with 10,000 fake reviews by randomly connect fake users and fake products, and 10,000 camouflage reviews by randomly connect fake users and real products. We adapt MDBP using the approach outlined in (Yu and Long, 2023) to identify the top 2000 k -MDBs on the attacked graph, treating all users and products in these k -MDBs as fake. For comparison, we implement four baseline methods: maximum biclique (Lyu et al., 2020), maximum k -biplex (Yu and Long, 2023) and (α, β) -core (Cerinšek and Batagelj, 2015), where the (α, β) -core is the largest subgraph in which the vertex degrees in left and right side are no less than α and β , respectively. To ensure fairness, we identify the top 2000 largest maximum biclique and maximum k -biplex as fake items. We also incorporate a widely used fraud detection method, FRAUDAR (Hooi et al., 2016), which is a densest-subgraph-based algorithm that identifies suspicious blocks of nodes by iteratively removing edges with the lowest likelihood of being part of a fraudulent structure, thereby resulting in several suspicious subgraphs.

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

and n from 1 to 7, where we fix $\sigma_U = p = 5$. The results are shown in Fig. 10a. As illustrated, when $\theta_V \geq 5$, 2-MDB consistently outperforms other methods. Additionally, the F1-score of 2-MDB rises over 1-MDB while 2-biplex drops below 1-biplex, as k -MDB offers finer-grained density relaxation than k -biplex with increasing k . For $\theta_V \leq 4$, biclique and FRAUDAR outperform k -MDB and k -biplex, as the latter two are too sparse and skewed to provide meaningful structure. However, the F1-score of biclique declines when $\theta_V \geq 5$, as the strict definition of biclique limits its ability to detect fake items, resulting in lower recall. In contrast, k -MDB and k -biplex, with their more relaxed definitions, still maintain high F1-scores.

We also evaluate the running time and the quality of subgraphs generated by each method, utilizing the widely adopted modularity ($\frac{1}{2m} \sum_{u \in U_S, v \in V_S} (1 - \frac{d(u)d(v)}{2m})$) metric, which quantifies the strength of internal connectivity relative to a random distribution and serves as a key indicator for assessing the community structure within the subgraph. Fig. 10b illustrates the optimal performance of each method across all parameter settings. As observed, 2-MDB achieves the highest modularity value, indicating the high quality of the subgraphs it produces, which consequently results in a higher F1 score for fraud detection. Additionally, although 1-biplex has a F1-score close to 2-MDB, its running time is significantly higher than that of 2-MDB. These results demonstrate the high effectiveness and efficiency of our solutions for fraud detection applications.



Method	F1	Modularity	Time
1-MDB	0.82	0.33	0.
2-MDB	0.95	0.45	15.
1-biplex	0.92	0.37	35
2-biplex	0.78	0.24	1187.
biclique	0.57	0.15	0.
(α, β)-core	0.45	0.04	0
FRAUDAR	0.72	0.27	17.

(a) F1-score with varying parameters

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

6. Related Work

Maximum k -defective clique search. As a problem related to our work, maximum k -defective clique search has received considerable attention in recent years, resulting in the development of many solutions and algorithms (Stozhkov et al., [2022](#); Gschwind et al., [2018](#); Trukhanov et al., [2013](#); Luo et al., [2024](#); Jin et al., [2024](#); Chen et al., [2021b](#); Gao et al., [2022](#); Chang, [2024](#), [2023](#); Dai et al., [2024a](#)). Specifically, Trukhanov et al. (Trukhanov et al., [2013](#)) proposed the first exact algorithm for detecting a maximum k -defective clique with a Russian Doll search scheme. Subsequently, Gschwind et al. (Gschwind et al., [2018](#)) improved the algorithm with a new preprocessing technique and a better verification procedure. These algorithms are derived from more general clique relaxation search frameworks. There are many studies focusing on the maximum k -defective clique. Particularly, Chen et al. (Chen et al., [2021b](#)) proposed a branch-and-bound algorithm with a time complexity below $O^*(2^n)$, and introduced a coloring-based upper bound to further improve the efficiency. Gao et al. (Gao et al., [2022](#)) proposed another branch-and-bound algorithm with new vertex and edge reduction rules, achieving better practical performance. Chang (Chang, [2023](#)) proposed an algorithm using a non-fully-adjacent-first branching rule, achieving tighter time complexity than the solutions in (Chen et al., [2021b](#)). Subsequently, Chang (Chang, [2024](#)) further enhanced the algorithm with an ordering technique, resulting in better time complexity and experimental results. Dai et al. (Dai et al., [2024a](#)) proposed a new branch-and-bound algorithm equipped with a pivot-based branching technique and a series of optimization techniques, attaining the best time complexity and experimental results to date. Nevertheless, it is difficult to adapt these algorithms for the k -MDB search problem, which is primarily due to the different definitions of maximum.

Maximum biclique search. The maximum biclique search problem has been extensively studied in recent years (Feng et al., [2018](#); Shaham et al., [2016](#); Peeters, [2003](#); Sözdinler and Özturan, [2018](#); Gillis and Glineur, [2014](#); Lyu et al., [2020](#); Chen et al., [2021a](#); Yuan et al., [2015](#); Dawande et al., [2001](#)). These studies can be classified into three categories. The first category is maximum edge biclique search, which aims to find a biclique with maximum number of edges. Peeters et al. (Peeters, [2003](#)) proved the NP-hardness of the problem, and Ambühl et al. (Ambühl et al., [2011](#)) showed the inapproximability of the problem. Shaham et al. (Shaham et al., [2016](#)) proposed a probabilistic algorithm for maximum edge biclique search utilizing a Monte-Carlo subspace clustering method.

Johnson, 1979) shown that a maximum vertex biclique can be found in polynomial time using a minimum-cut algorithm. The third category is maximum balanced biclique search, which intends to find a maximum biclique with an equal number of vertices on both sides. Yuan et al. (Yuan et al., 2015) proposed an evolutionary algorithm for the problem with structure mutation. Wang et al. (Wang et al., 2018) proposed a local search framework with four optimization heuristics. Zhou et al. (Zhou and Hao, 2019) proposed an upper bound propagation served for two algorithms, which are respectively based on branch-and-bound and integer linear programming. Chen et al. (Chen et al., 2021a) introduced two efficient branch-and-bound algorithms utilizing a bipartite sparsity measurement, targeting small dense and large sparse graphs, achieving improved time complexities and better practical performance. All these mentioned techniques are tailored for maximum biclique search and cannot be directly used to solve our k -MDB search problem.

7. Conclusion

In this paper, we investigate the problem of finding a maximum k -defective biclique in a bipartite graph. We propose two novel algorithms to tackle the problem. The first algorithm employs a newly-designed binary branching strategy that reduces the time complexity to below $O^*(2^n)$, while the second algorithm utilizes a novel pivoting-based branching strategy, achieving a notably lower time complexity. Additionally, we propose a series of nontrivial optimization techniques to further improve the efficiency of our algorithms. Extensive experiments conducted on 10 real-word graphs demonstrate the notable effectiveness and efficiency of our algorithms and optimization techniques.

References

(1)

RNO (line)

Online.
Maximum Defective Biclique Search in Large Bipartite Graphs (full version).

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

Liu, 2020.

Pivot-based Maximal Biclique Enumeration. In
IJCAI. 3558–3564.

Alexe et al. (2004)

Gabriela Alexe, Sorin Alexe, Yves Crama,
Stephan Foldes, Peter L Hammer, and Bruno
Simeone. 2004.

Consensus algorithms for the generation of all
maximal bicliques.

Discrete Applied Mathematics 145, 1 (2004),
11–21.

Alzahrani and Horadam (2019)

Taher Alzahrani and Kathy Horadam. 2019.

Finding maximal bicliques in bipartite
networks using node similarity.

Applied Network Science 4, 1 (2019), 21.

Ambühl et al. (2011)

Christoph Ambühl, Monaldo Mastrolilli, and
Ola Svensson. 2011.

Inapproximability results for maximum edge
biclique, minimum linear arrangement, and
sparsest cut.

SIAM J. Comput. 40, 2 (2011), 567–596.

Braun et al. (2017)

Fabian Braun, Olivier Caelen, Evgueni N.
Smirnov, Steven Kelk, and Bertrand Lebichot.
2017.

Improving Card Fraud Detection Through
Suspicious Pattern Discovery. In *Advances in
Artificial Intelligence: From Theory to Practice*,
Salem Benferhat, Karim Tabia, and Moonis Ali
(Eds.). Springer International Publishing, 181–
190.

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

Chang (2023)

Lijun Chang. 2023.
Efficient Maximum k -Defective Clique
Computation with Improved Time Complexity.
*Proceedings of the ACM on Management of
Data* 1, 3 (2023), 1–26.

Chang (2024)

Lijun Chang. 2024.
Maximum Defective Clique Computation:
Improved Time Complexities and Practical
Performance.
arXiv preprint arXiv:2403.07561 (2024).

Chen et al. (2021a)

Lu Chen, Chengfei Liu, Rui Zhou, Jiajie Xu, and
Jianxin Li. 2021a.
Efficient exact algorithms for maximum
balanced biclique search in bipartite graphs. In
*Proceedings of the 2021 International
Conference on Management of Data*. 248–
260.

Chen et al. (2022)

Lu Chen, Chengfei Liu, Rui Zhou, Jiajie Xu, and
Jianxin Li. 2022.
Efficient maximal biclique enumeration for
large sparse bipartite graphs.
Proceedings of the VLDB Endowment 15, 8
(2022), 1559–1571.

Chen et al. (2021b)

Xiaoyu Chen, Yi Zhou, Jin-Kao Hao, and
Mingyu Xiao. 2021b.
Computing maximum k -defective cliques in
massive graphs.

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

[Why HTML?](#) [Report Issue](#) [Back to Abstract](#) [Download PDF](#)

Theoretically and Practically Efficient
Maximum Defective Clique Search.

Proceedings of the ACM on Management of Data 2, 4 (2024), 1–27.

Dai et al. (2024b)

Qiangqiang Dai, Rong-Hua Li, Donghang Cui, Meihao Liao, Yu-Xuan Qiu, and Guoren Wang. 2024b.

Efficient Maximal Biplex Enumerations with Improved Worst-Case Time Guarantee.

Proceedings of the ACM on Management of Data 2, 3 (2024), 1–26.

Dai et al. (2023a)

Qiangqiang Dai, Rong-Hua Li, Meihao Liao, and Guoren Wang. 2023a.

Maximal defective clique enumeration.

Proceedings of the ACM on Management of Data 1, 1 (2023), 1–26.

Dai et al. (2023b)

Qiangqiang Dai, Rong-Hua Li, Xiaowei Ye, Meihao Liao, Weipeng Zhang, and Guoren Wang. 2023b.

Hereditary Cohesive Subgraphs Enumeration on Bipartite Graphs: The Power of Pivot-based Approaches.

Proceedings of the ACM on Management of Data 1, 2 (2023), 1–26.

Dawande et al. (2001)

Milind Dawande, Pinar Keskinocak, Jayashankar M. Swaminathan, and Sridhar R. Tayur. 2001.

On Bipartite and Multipartite Clique Problems.

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

A comparative analysis of clustering
algorithms for gene expression data.
Briefings in bioinformatics 14, 3 (2013), 279–
292.

Feng et al. (2018)

Qilong Feng, Shaohua Li, Zeyang Zhou, and
Jianxin Wang. 2018.
Parameterized algorithms for edge biclique
and related problems.
Theoretical Computer Science 734 (2018),
105–118.

Fomin and Kaski (2013)

Fedor V Fomin and Petteri Kaski. 2013.
Exact exponential algorithms.
Commun. ACM 56, 3 (2013), 80–88.

Gao et al. (2022)

Jian Gao, Zhenghang Xu, Ruizhi Li, and
Minghao Yin. 2022.
An exact algorithm with new upper bounds for
the maximum k -defective clique problem in
massive sparse graphs. In *Proceedings of the
AAAI Conference on Artificial Intelligence*,
Vol. 36. 10174–10183.

Garey and Johnson (1979)

Michael R Garey and David S Johnson. 1979.
Computers and intractability. Vol. 174.
freeman San Francisco.

Gillis and Glineur (2014)

Nicolas Gillis and François Glineur. 2014.
A continuous characterization of the
maximum-edge biclique problem.

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

[Why HTML?](#) [Report Issue](#) [Back to Abstract](#) [Download PDF](#)

Maximum weight relaxed cliques and Russian Doll Search revisited.
Discrete Applied Mathematics 234 (2018), 131–138.

Hooi et al. (2017)

Bryan Hooi, Kijung Shin, Hyun Ah Song, Alex Beutel, Neil Shah, and Christos Faloutsos.
2017.
Graph-Based Fraud Detection in the Face of Camouflage.
ACM Trans. Knowl. Discov. Data 11, 4 (2017), Article 44.

Hooi et al. (2016)

Bryan Hooi, Hyun Ah Song, Alex Beutel, Neil Shah, Kijung Shin, and Christos Faloutsos.
2016.
Fraudar: Bounding graph fraud in the face of camouflage. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*. 895–904.

Jin et al. (2024)

Mingming Jin, Jiongzhi Zheng, and Kun He.
2024.
KD-Club: An Efficient Exact Algorithm with New Coloring-based Upper Bound for the Maximum k -Defective Clique Problem. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 20735–20742.

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

[Why HTML?](#) [Report Issue](#) [Back to Abstract](#) [Download PDF](#)

Proceedings of the 41st Annual Hawaii International Conference on System Sciences (HICSS 2008)(HICSS), Vol. 1. 473.

Lehmann et al. (2008)

Sune Lehmann, Martin Schwartz, and Lars Kai Hansen. 2008.

Biclique communities.

Physical Review E—Statistical, Nonlinear, and Soft Matter Physics 78, 1 (2008), 016108.

Ley (2002)

Michael Ley. 2002.

The DBLP Computer Science Bibliography: Evolution, Research Issues, Perspectives (*String Processing and Information Retrieval*). Springer Berlin Heidelberg, 1–10.

Li et al. (2023)

Jingdong Li, Zhao Li, Xiaoling Wang, Xingjian Lu, Ji Zhang, and Hongyang Chen. 2023.

GPU-Accelerated Maximal Bicliques Mining Framework for Large E-commerce Networks. 539-544 pages.

Li et al. (2007)

Jinyan Li, Guimei Liu, Haiquan Li, and Limsoon Wong. 2007.

Maximal biclique subgraphs and closed pattern pairs of the adjacency matrix: A one-to-one correspondence and mining algorithms.

IEEE Transactions on Knowledge and Data Engineering 19, 12 (2007), 1625–1637.

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

International Conference on Data Mining.
SIAM, 72–83.

Liu et al. (2006)

Guimei Liu, Kelvin Sim, and Jinyan Li. 2006.
Efficient mining of large maximal bicliques. In
Data Warehousing and Knowledge Discovery: 8th International Conference, DaWaK 2006, Krakow, Poland, September 4-8, 2006. Proceedings 8. Springer, 437–448.

Luo et al. (2024)

Chunyu Luo, Yi Zhou Zhengren Wang, and
Mingyu Xiao. 2024.
A Faster Branching Algorithm for the
Maximum k -Defective Clique Problem.
arXiv preprint arXiv:2407.16588 (2024).

Lyu et al. (2020)

Bingqing Lyu, Lu Qin, Xuemin Lin, Ying Zhang,
Zhengping Qian, and Jingren Zhou. 2020.
Maximum biclique search at billion scale.
Proceedings of the VLDB Endowment (2020).

Madeira and Oliveira (2004)

Sara C Madeira and Arlindo L Oliveira. 2004.
Biclustering algorithms for biological data
analysis: a survey.
*IEEE/ACM transactions on computational
biology and bioinformatics* 1, 1 (2004), 24–45.

Peeters (2003)

René Peeters. 2003.
The maximum edge biclique problem is NP-
complete.

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

[Why HTML?](#) [Report Issue](#) [Back to Abstract](#) [Download PDF](#)

Wilhelm Gruissem, Lars Hennig, Lothar Thiele, and Eckart Zitzler. 2006.

A systematic comparison and evaluation of biclustering methods for gene expression data. *Bioinformatics* 22, 9 (2006), 1122–1129.

Shaham et al. (2016)

Eran Shaham, Honghai Yu, and Xiao-Li Li. 2016.

On finding the maximum edge biclique in a bipartite graph: a subspace clustering approach. In *Proceedings of the 2016 SIAM International Conference on Data Mining*. SIAM, 315–323.

Shahinpour et al. (2017)

Shahram Shahinpour, Shirin Shirvani, Zeynep Ertem, and Sergiy Butenko. 2017.

Scale reduction techniques for computing maximum induced bicliques. *Algorithms* 10, 4 (2017), 113.

Stozhkov et al. (2022)

Vladimir Stozhkov, Austin Buchanan, Sergiy Butenko, and Vladimir Boginski. 2022.

Continuous cubic formulations for cluster detection problems in networks. *Mathematical Programming* 196, 1 (2022), 279–307.

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

Why Report Back to Download
HTML? Issue Abstract PDF

IEEE, 132–137.

Trukhanov et al. (2013)

Svyatoslav Trukhanov, Chitra Balasubramaniam, Balabhaskar Balasundaram, and Sergiy Butenko. 2013. Algorithms for detecting optimal hereditary structures in graphs, with application to clique relaxations. *Computational Optimization and Applications* 56 (2013), 113–130.

Wang et al. (2023)

Jianhua Wang, Jianye Yang, Ziyi Ma, Chengyuan Zhang, Shiyu Yang, and Wenjie Zhang. 2023. Efficient Maximal Biclique Enumeration on Large Uncertain Bipartite Graphs. *IEEE Trans. on Knowl. and Data Eng.* 35, 12 (2023), 12634–12648.
[doi:10.1109/tkde.2023.3272110](https://doi.org/10.1109/tkde.2023.3272110)

Wang et al. (2021)

Kai Wang, Yiheng Hu, Xuemin Lin, Wenjie Zhang, Lu Qin, and Ying Zhang. 2021. A Cohesive Structure Based Bipartite Graph Analytics System. 4799–4803 pages.

Wang et al. (2022)

Kai Wang, Wenjie Zhang, Xuemin Lin, Lu Qin, and Alexander Zhou. 2022. Efficient personalized maximum biclique search. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 498–511.

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

[Why HTML?](#) [Report Issue](#) [Back to Abstract](#) [Download PDF](#)

- Yu et al. (2006) Haiyuan Yu, Alberto Paccanaro, Valery Trifonov, and Mark Gerstein. 2006. Predicting interactions in protein networks by completing defective cliques. *Bioinformatics* 22, 7 (2006), 823–829.
- Yu and Long (2023) Kaiqiang Yu and Cheng Long. 2023. Maximum k -Biplex Search on Bipartite Graphs: A Symmetric-BK Branching Approach. *Proc. ACM Manag. Data* 1, 1 (2023), Article 49.
- Yu et al. (2022) Kaiqiang Yu, Cheng Long, Shengxin Liu, and Da Yan. 2022. Efficient Algorithms for Maximal k -Biplex Enumeration. In *Proceedings of the 2022 International Conference on Management of Data*. 860–873.
- Yuan et al. (2015) Bo Yuan, Bin Li, Huanhuan Chen, and Xin Yao. 2015. A New Evolutionary Algorithm with Structure Mutation for the Maximum Balanced Biclique Problem. *IEEE Transactions on Cybernetics* 45, 5 (2015), 1054–1067.
- Zhou et al. (2025) Hongru Zhou, Shengxin Liu, and Ruidi Cao. 2025. Efficient Maximum Vertex (k,l) -Biplex Computation on Bipartite Graphs.

This is **experimental HTML** to improve accessibility.
We invite you to report rendering errors. Learn more
about this project and help improve conversions.

[Why HTML?](#) [Report Issue](#) [Back to Abstract](#) [Download PDF](#)

add search with graph reduction for finding
maximum balanced bicliques in bipartite
graphs.

Eng. Appl. Artif. Intell. 77 (2019), 86–97.